



HERIOT-WATT UNIVERSITY

FINAL YEAR PROJECT

**Discrete notions of curvature on graphs:
Ollivier Ricci Curvature and
non-convergence of the Ollivier Ricci Flow**

Maria Esteban Casadevall

H00325269

supervised by
Dr. Alessandro Sisto

March 22, 2024

Contents

1	Introduction	5
2	Ollivier Ricci Curvature	6
2.1	Construction	6
2.2	Connection to Ricci Curvature in a manifold	8
2.3	Graphs with constant curvature	10
2.3.1	Regular Graphs of constant curvature	12
3	Ollivier Ricci Flow	14
3.1	Construction	15
3.2	Convergence of Ricci Flow	16
3.2.1	Non-Convergence of the Ollivier Ricci Flow	19
3.2.2	Experimental Analysis	19
3.2.3	Proof of Theorem 1	21
3.2.4	Interesting points and future work on the convergence of the Ricci Flow .	28
4	Robustness of Ollivier Ricci Curvature and Ricci Flow	29
5	Conclusion	31
6	Appendix	33
6.1	Appendix I: Regular graphs of constant curvature up to 11 vertices	33
6.2	Appendix II: Random graphs with constant curvature	35
6.3	Appendix III: Experimental analysis of non-convergence	35
6.4	Appendix IV: Robustness analysis	36

Acknowledgements

I would like to thank my supervisor, Alessandro Sisto, for all his support throughout this project and for encouraging me to follow my interest on the topic. I learnt a lot from his guidance and expertise, which has not only helped me enormously on this thesis but will undoubtedly help me in my future research experiences.

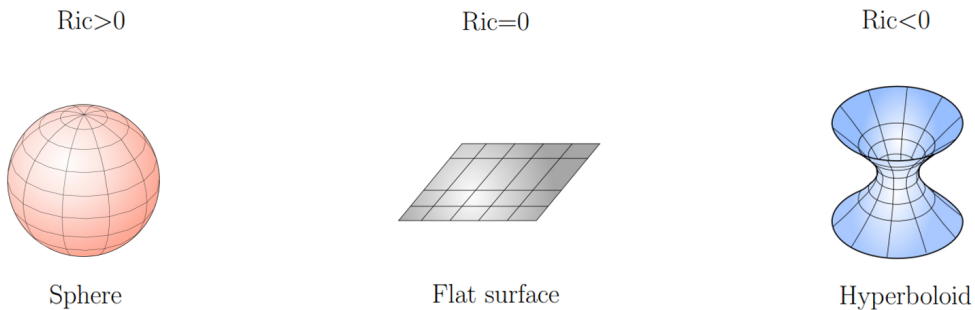
Abstract

This dissertation focuses on the Ollivier Ricci curvature on a graph. This is a discrete notion of edge-based curvature that can be formulated in terms of an optimal transport problem. The first part of the report introduces the definition and discusses some of its main properties. The second part of the report focuses on the Ollivier Ricci Flow, an iterative process which uses the values of the curvature to try to find a set of edge weights that make the curvature of the graph constant. We present a graph for which this algorithm doesn't converge and explain experimentally and theoretically the reason why. Finally, we study the robustness of the Ollivier Ricci curvature and the Ollivier Ricci Flow to noisy data inputs using simulations in Python.

1 Introduction

Complex networks are now fundamental objects in many scientific fields. They can be found in everything from financial markets, roads and layout of cities, drug discovery, social networks and more. These types of datasets can be mathematically phrased as a set of nodes, items, connected together by edges. Significant effort has been put to develop novel and powerful techniques to analyse these objects. A prominent approach has been to use geometrical information of the network, such as curvature, to understand its underlying structure.

Curvature is usually a notion found on the context of differential manifolds. There are several notions of curvature that can be defined at a point, some of them being more complex and sophisticated than others, but they all aim to quantify the deviation of an object from being flat. Typically the values are defined to be large when the object deviates strongly from its flat counterpart, and to be close to zero when it is nearly flat. In the image below there is an example of surfaces with positive, zero and negative Ricci curvature.



Discrete spaces don't have a differential structure, but recent research has shown that the concept of curvature can be transferred to the discrete setting by using *analogies* with the continuous case. There are multiple notions of discrete curvature that have been defined in the last few years, each of them revealing different properties of the network. This dissertation focuses on the Ollivier Ricci Curvature. This is an edge based curvature that can be formulated in terms of an optimal transportation problem. This curvature has received special attention due to its success in identifying community structures within a graph. Broadly speaking, edges with positive curvature tend to represent edges within a cluster, while negatively curved edge tend to be the *bridges* within communities. This property potentially allows to identify communities within a graph using intrinsic properties of the network only, without the need of any embedding techniques.

The first part of this report focuses on the Ollivier Ricci Curvature. We provide its definition and explain its connection to the Ricci Curvature defined in manifolds. We then expose some experimental results obtained regarding regular graphs with constant curvature. On the second part we introduce the Ollivier Ricci Flow, an iterative process that aims to *flatten* the curvature of a graph. We explain the methodology behind the algorithm and show an example of the Ollivier Ricci Flow implemented in a real-world dataset. We then focus on the study of the convergence of this algorithm. We prove experimentally and theoretically that the algorithm applied to a graph consisting of two triangles glued together by an edge doesn't converge. To conclude the dissertation, we provide some potential future lines of research.

2 Ollivier Ricci Curvature

2.1 Construction

Definition 1: Let $G(V, E)$ be a connected graph. A *probability distributions* μ on G can be understood as a function $\mu : V \rightarrow [0, 1]$ with

$$\sum_{x \in V} \mu(x) = 1$$

Let $G = (V, E)$ be a finite undirected graph with vertex set V , edge set E , and with no multiple edges or self loops. The Ollivier Ricci Curvature is an edge based curvature that can be formulated in terms of the optimal transport problem between probabilities distributions around adjacent nodes. The probability distribution centered at a node x is defined as:

$$m_x^\alpha(y) = \begin{cases} \alpha & \text{if } y = x \\ \frac{(1-\alpha)\psi(\omega_{xy})}{\sum_{z \in N_x} \psi(\omega_{zx})} & \text{if } y \in N_x \\ 0 & \text{Otherwise} \end{cases} \quad (1)$$

where:

- N_x (neighborhood of x) is the set of nodes that are adjacent to x .
- $\psi : \mathbb{R} \rightarrow \mathbb{R}$ is a one-to-one function in terms of the edge weight. For an unweighted graph, this term can be disregarded (gets cancelled out with the normalization factor). For a weighted graph, ψ determines how much does the edge weight impact the mass distribution at node x . Intuitively, if we look at the edge weight as a measure of proximity between adjacent nodes, we would want to put less mass on the nodes that are more *far away* from x (connected with a high edge weight). Therefore, ψ needs to be a decreasing function for our purposes. This term will be of special importance when implementing the Ricci Flow described in Section 3.
- The parameter $\alpha \in [0, 1]$ is called the *idleness* as can be seen as the probability of a random walker to stay at x . As we would see in the upcoming sections, this parameter will play an important role on the convergence of the Ollivier Ricci Flow.

We can look at the probability measure defined in (1) as a mass distribution, which needs to be transported to the mass distribution defined around the other adjacent node. We thus produce a transport plan ξ that satisfies the following conditions:

$$\begin{aligned} \sum_{l \in N_y} \xi_{xy}(k, l) &= m_x^\alpha(k) \\ \sum_{k \in N_x} \xi_{xy}(k, l) &= m_y^\alpha(l) \end{aligned}$$

The first condition ensures that the total amount send from node k to all of the nodes in the neighbor of y is equal to the amount initially at k . The second condition ensures that the total amount send from all the nodes neighboring x to a node l in the neighborhood of y , is be equal to the final amount at l . Let Π be the set of all transport plans from $m_x^\alpha(v)$ to $m_y^\alpha(v)$. The distance between the probability distribution can be defined as

$$W[m_x^\alpha, m_y^\alpha] = \inf_{\xi \in \Pi} \sum_{k \in N_x, l \in N_y} d(k, l) \xi_{xy}(k, l) \quad (2)$$

Where W is the Wasserstein distance (also known as the Earth-mover distance). Finally, the Ricci curvature of an edge xy is defined as [1]:

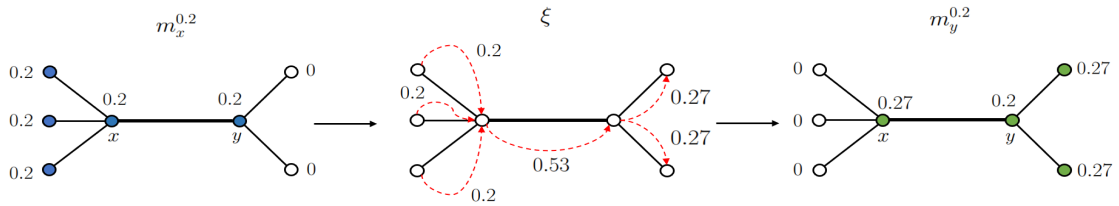
$$k_{xy}^\alpha = 1 - \frac{W[m_x^\alpha, m_y^\alpha]}{d(x, y)} \quad (3)$$

The Wasserstein distance can also be expressed as a linear programming problem, where ρ_{ij} is the proportion of mass originally at x_i transported to y_j :

$$\begin{aligned} \min_x \quad & \sum_{j=1}^n \sum_{i=1}^m d(x_i, y_j) \rho_{ij} m_x^\alpha(x_i) \\ \text{s.t.} \quad & \sum_{j=1}^n \rho_{ij} = 1 \quad \forall i \\ & \sum_{i=1}^m \rho_{ij} m_x^\alpha(x_i) = m_y^\alpha(y_j) \quad \forall j \\ & 0 \leq \rho_{ij} \leq 1 \end{aligned} \quad (4)$$

Note that (4) is bounded from below as $W[m_x^\alpha, m_y^\alpha] \geq 0$ and is not an unfeasible as in a connected graph there always exist a path between two nodes x_i and y_j . Therefore, by the *Fundamental Theorem of Linear Programming*, there exists an optimal solution to (4) (but it doesn't need to be unique). Solving this optimization problem can be very computationally expensive, especially for large graphs with high number of edges, as is the case of real-world social networks. Therefore, in practice different approximations are used to reduce the complexity of the calculations.

Example: Let's consider the following unweighted graph. We want to calculate the curvature along edge xy . The mass distributions around node x and y is represented in the diagram below (left and right diagram, respectively):



As this is a simple graph, it's quite easy to find the optimal transport plan ξ between m_x^α and m_y^α . This consists on moving all the mass from the nodes adjacent to x (apart from node y) to x and leaving 0.27 mass there. We then transport the rest of the mass, 0.53, along edge xy and leave 0.2 mass at node y . Finally, we equidistribute the remaining mass between the rest of the neighbours of node y .

Therefore,

$$\begin{aligned} W[m_x^{0.2}, m_y^{0.2}] &= 0.2 \cdot 3 + 0.53 + 0.27 \cdot 2 = 1.4 \\ k_{xy}^{0.2} &= 1 - 1.4 = -0.4 \end{aligned}$$

2.2 Connection to Ricci Curvature in a manifold

In this subsection, we provide an intuition on the connection between the Ollivier Ricci Curvature and the Ricci Curvature defined in Riemannian manifolds. We will not provide the formal definition of Ricci Curvature in a manifold, but we will introduce one of its main characterisations: Let x and y denote two close points in a manifold $\mathcal{M}$ of dimension n , and let $d(x,y)$ be the distance along a geodesic γ joining these two points. We can then take a vector $\mathbf{v} \in T_x\mathcal{M}$ and parallel transport it along γ to a vector $\mathbf{v}' \in T_y\mathcal{M}$. Informally, to parallel transport a vector along a curve means to transport it without changing its magnitude and direction (We would need to introduce the concept of Levi-Civita connection for a formal definition). We then project x along $\mathbf{v}$ back to the manifold again to obtain x' and project y along $\mathbf{v}'$ to obtain y' , and calculate the distance between these two points along a geodesic. We repeat this procedure for every possible direction of v and v' , constructing an infinitesimal balls $B_\epsilon(x)$ and $B_\epsilon(y)$ centered at x and y respectively. It is well known that the average distance between x and y' is $d(x,y)(1 - \epsilon^2 \frac{\text{Ric}(\mathbf{v},\mathbf{v})}{2n} + \mathcal{O}(\epsilon^3 + \epsilon^2 d(x,y)))$. [2]

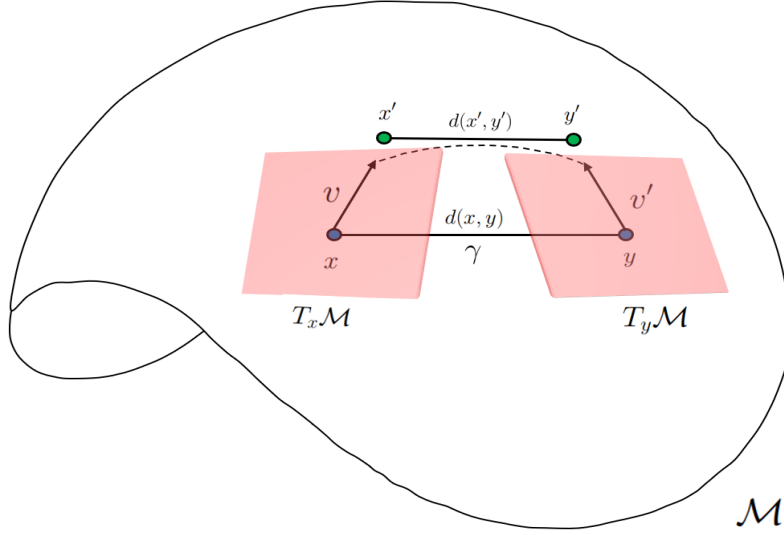


Figure 1: Ricci Curvature on a manifold

In the Ollivier Ricci Curvature, we replace the the idea of infinitesimal balls $B_\epsilon(x)$ and $B_\epsilon(y)$ with the mass distributions m_x^α and m_y^α . The distance between these masses distributions is then computed using the Wasserstein distance from optimal transport theory, and is compared to the distance between the centers of the ball, that is, $d(x,y)$ in formula (3). Therefore, the connection between the Ricci curvature in a manifold and the Ollivier Ricci Curvature is that both notions reflect the fact that when the the curvature is positive, the average distance between respective points in small metric ball is smaller than the distance between its centers (and the opposite for negative curvature).

$$\text{Average distance } d(x', y') = d(x, y)(1 - \epsilon^2 \frac{\text{Ric}(\mathbf{v}, \mathbf{v})}{2n}) + \mathcal{O}(\epsilon^3 + d(x, y)\epsilon^2)$$

$$k_{xy}^\alpha = 1 - \frac{W[m_x^\alpha, m_y^\alpha]}{d(x, y)}$$

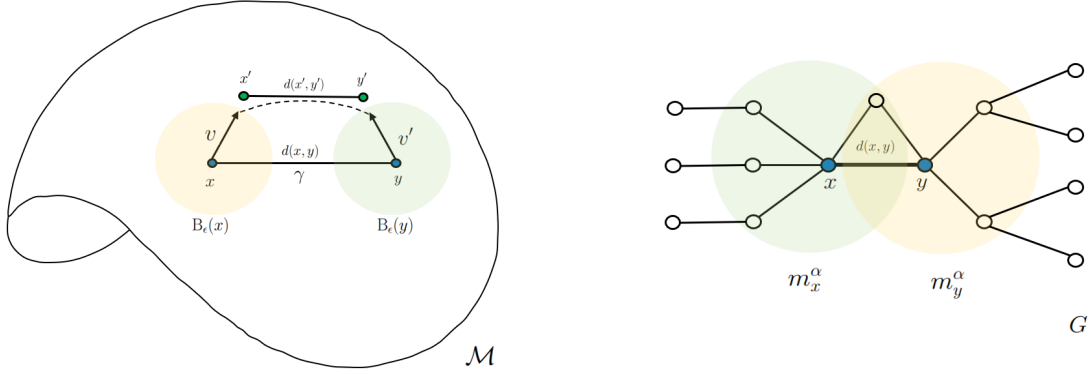


Figure 2: Comparison between Ricci Curvature defined on manifolds (left) and Ricci Curvature defined on graphs (right)

From this construction we can expect that graphs with a clique-like structure will have a positive curvature as the mass distribution around adjacent nodes will highly overlap, so $W[m_x^\alpha, m_y^\alpha]$ will be small and, hence, often less than 1. By contrast, graphs with a tree-like structure will have a negative curvature as we will have to transport large amounts of mass to go from m_x^α to m_y^α , so $W[m_x^\alpha, m_y^\alpha]$ will tend to be larger than 1. On the figure below we can see a summary of these properties.

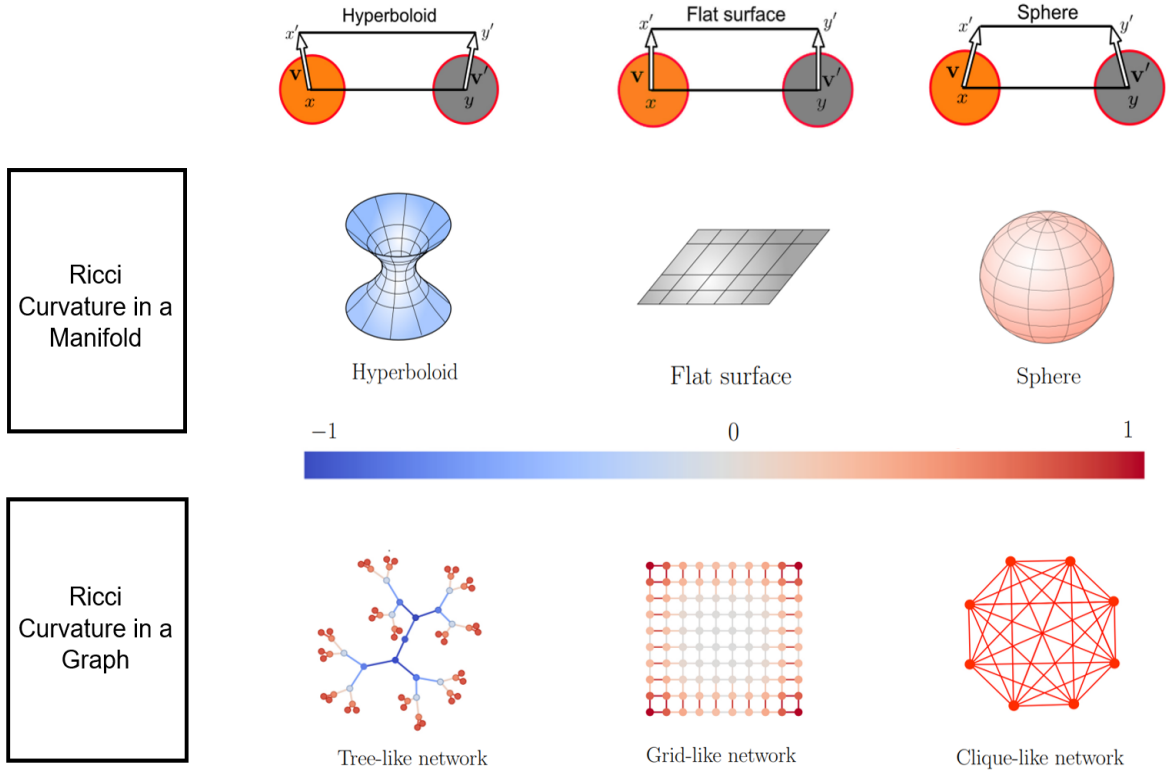


Figure 3: Ricci Curvature on manifolds vs Ricci Curvature on graphs [3]

2.3 Graphs with constant curvature

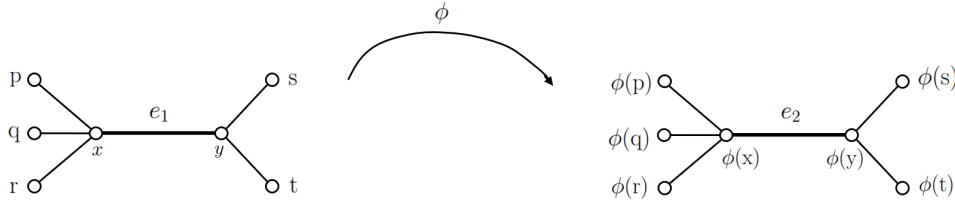
In this subsection we study graphs of constant curvature, that is, graphs whose all edges have the same value of curvature. The study of these graphs is of special interest as their shape is unchanged under the Ricci Flow, as we will describe in the next section. In this dissertation, we have written down the proof for the case of edge-transitive graphs, cycles and complete graphs. The following definitions were used on the proofs:

Definition 2: The girth of an undirected graph is the length of a shortest cycle contained in the graph.

Definition 3: A graph G is edge-transitive if for all pairs (e_1, e_2) there exists an element $\psi \in \text{Aut}(G)$ such that $\psi(e_1) = e_2$.

Proposition 1 Edge transitive graphs have constant curvature

Proof: Let ϕ be an automorphism of G , $\phi : V(G) \rightarrow V(G)$, such that $xy \in E(G)$ if and only if $\phi(x)\phi(y) \in E(G)$.



By definition, ϕ preserves node adjacency. That is, if $s \sim x$ if and only if $\phi(s) \sim \phi(x)$. As a consequence, $\deg(x) = \deg(\phi(x))$ and the mass distribution around node x , is the same as the mass distribution around $\phi(x)$. So $m_x^\alpha = m_{\phi(x)}^\alpha$. In addition, as the node adjacency is preserved, the shortest path between x and y is the same as the shortest path between $\phi(x)$ and $\phi(y)$. As a consequence, $d(x, y) = d(\phi(x), \phi(y))$. Putting these statements together, it follows that $W[m_x^\alpha, m_y^\alpha] = W[m_{\phi(x)}^\alpha, m_{\phi(y)}^\alpha]$.

$$\begin{aligned} W[m_x^\alpha, m_y^\alpha] &= \inf_{\xi \in \Pi} \sum_{k \in N_x, l \in N_y} d(k, l) \xi_{xy}(k, l) = \\ &= \inf_{\xi \in \Pi} \sum_{k \in N_{\phi(x)}, l \in N_{\phi(y)}} d(\phi(k), \phi(l)) \xi_{\phi(x)\phi(y)}(\phi(k), \phi(l)) = W[m_{\phi(x)}^\alpha, m_{\phi(y)}^\alpha] \end{aligned}$$

As there exists an automorphism ϕ that maps any two pairs of adjacent nodes to each other, we then conclude that the curvature in every edge will be the same.

Proposition 2 Complete graphs have positive constant curvature of $k_{xy}^\alpha = 1 - \left| \alpha - \frac{1-\alpha}{N-1} \right|$.

Proof: Let G be a complete graph with N nodes. By definition, all the nodes in a complete graph are connected to each other. Hence, the mass distribution around two adjacent nodes will overlap. In particular,

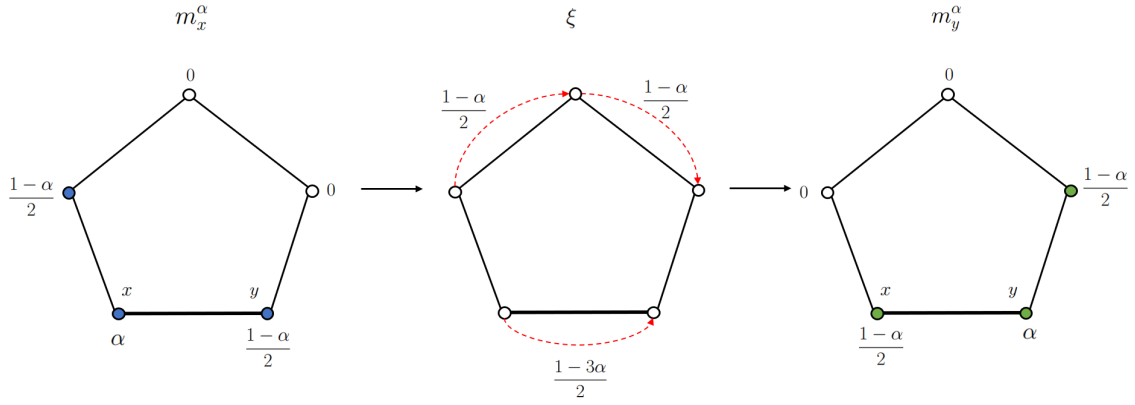
$$m_x^\alpha(v) = \begin{cases} \alpha & \text{if } v = x \\ \frac{1-\alpha}{N-1} & \text{otherwise} \end{cases} \quad m_y^\alpha(v) = \begin{cases} \alpha & \text{if } v = y \\ \frac{1-\alpha}{N-1} & \text{otherwise} \end{cases}$$

Hence, to go from $m_x^\alpha(y)$ to $m_x^\alpha(v)$ we just need to move $\left| \alpha - \frac{1-\alpha}{N-1} \right|$ along the edge xy . Therefore,

$$W[m_x^\alpha, m_y^\alpha] = \left| \alpha - \frac{1-\alpha}{N-1} \right| \rightarrow k_{xy}^\alpha = 1 - \left| \alpha - \frac{1-\alpha}{N-1} \right| > 0, \forall \alpha$$

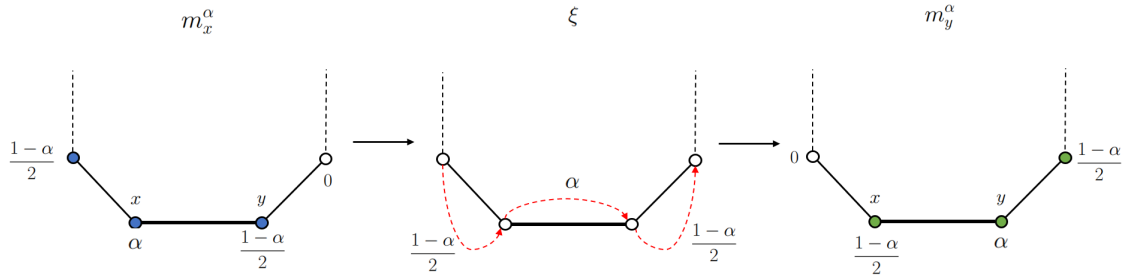
Proposition 3 Cycle graphs C_n of n vertices have constant curvature. In particular, for $n < 6$ the curvature $k_{xy}^\alpha > 0 \forall xy \in E(C_n)$, and for $n \geq 6$, $k_{xy}^\alpha = 0 \forall xy \in E(C_n)$.

Proof: Let's first consider cycles C_n for $n < 6$. The mass distribution around node x and y for $n=5$ is represented in the diagram below, along with the optimal transport plan ξ . For $n \neq 5$, the mass distribution would be the same, but the optimal transport plan would move $\frac{1-\alpha}{2}$ along $n-3$ edges.



$$W[m_x^\alpha, m_y^\alpha] = \left| \frac{1-3\alpha}{2} \right| + \frac{1-\alpha}{2} \cdot (n-3) > 0, \forall \alpha$$

For C_n for $n < 6$, the mass distribution around node x and y is represented in the diagram below, along with the optimal transport plan ξ . As the girth of the graph is larger than 3, the optimal transport plan ξ just shifts every mass through one edge.



$$W[m_x^\alpha, m_y^\alpha] = \frac{1-\alpha}{2} + \alpha + \frac{1-\alpha}{2} = 1$$

$$k_{xy}^\alpha = 1 - 1 = 0$$

All the edges xy of C_n will have the same transport plan ξ (as cycles are edge-transitive graphs), so the curvature will be the same in all the edges.

2.3.1 Regular Graphs of constant curvature

We have experimentally found regular connected graphs of constant curvature up to 11 vertices. We have used 2 different methodologies depending on the number of vertices and degree of the graph:

- **Graphs up to 10 vertices and degree 5:** We have used a database that contains the full list of regular graphs with those characteristics [4]. We have extracted their adjacency list and uploaded it into Python. We have created a Python function that returns those graphs whose edges have the same curvature. The Python code can be found in Appendix II.
- **Graphs with 11 vertices and 10 vertices with degree larger than 5:** The adjacency list mentioned above is not available for graphs larger than 10 vertices and degree 5. Therefore, we generated the regular graphs using a uniformly random generator algorithm in Python [5] and extracted the ones with constant curvature. To do that, we first need to determine the minimum number of iterations needed to make sure we have generated most of the graphs in the sample space.

Let X be the number of distinct graphs generated after n iterations. In particular,

- Let $X_i = 1$ if a graph G is generated at least once
- Let $X_i = 0$ otherwise

The algorithm we used generates graphs asymptotically uniform, we can express the expected value X as follows:

$$E(X) = E\left(\sum_{i=1}^m X_i\right) = \sum_{i=1}^m \mathbf{P}(X_i = 1) = \sum_{i=1}^m \left(1 - \left(\frac{m-1}{m}\right)^n\right) = m \left(1 - \left(\frac{m-1}{m}\right)^n\right)$$

To get $E(X)$ we then need to determine the size of the sample space m . We know that there are 265 and 266 distinct graphs with 11 vertices and degree 4 and 6, respectively (from source [4]). Nevertheless, the sample space m is larger than that because the algorithm is generating isomorphic graphs as well. Therefore, to calculate the size of the sample space m , we have calculated the average number of isomorphic graphs that exist for every graph G . To do that, we have used the VF2 Algorithm in Python to calculate the size of the automorphism group of a random generated graph G , over 1000 iterations, to obtain the average size of $|\text{Aut}(G)|$. Therefore, the sample space m can be approximated as:

$$m \approx \text{Number of distinct graphs} \times \sum_{i=1}^{1000} \frac{|\text{Aut}(G)|}{1000}$$

Putting these results together, for $E(x) \geq 0.9$ we need at least 1100 iterations. We now can check for the graphs of constant curvature over those generated.

The results containing all regular graphs of constant curvature can be found in Appendix I.

Notice that we have found a regular graph of constant curvature for every combination of possible vertex and degrees. As future work, it would be interesting to study if this is the case for larger regular graphs. The methodology used above using random graphs generators can also be used to find graphs of constant curvature for higher number of vertices. For the purpose of this dissertation, we have only analyzed up to 11 vertices due to the high computational cost of calculating the Ollivier Ricci Curvature, mainly computing the Wasserstein distance, for such big number of graphs.

Throughout this analysis, we have also found the following interesting results:

1. Regular graph with negative constant curvature: All the graphs listed in Appendix I have positive constant curvature. This is because for small connected graphs, the amount of mass we need to transport from adjacent mass distribution is not enough to have $W[m_x^\alpha, m_y^\alpha] > 1$. The left graph in Figure 4 is one example of a graph with constant negative curvature. Our intuition is that as girth of the regular graph increases, more examples of constant negative curvature will appear.
2. Regular graph with constant curvature but not edge transitive: All the graphs listed in Appendix I are edge transitive. We have only found one example of a regular graph with constant curvature which is not edge transitive. This graph is shown in Figure 4 on the right.

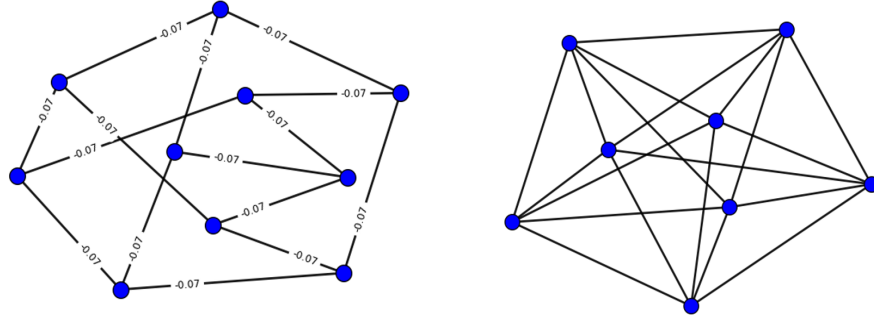


Figure 4: Graph with constant negative curvature (left) and graph with constant curvature which is not edge-transitive (right)

3 Ollivier Ricci Flow

The Ollivier Ricci Flow is an iterative process that modifies the edge weights to try to flatten the curvature of the graph. Broadly speaking, edges with positive curvature will shrink (the edge weight will decrease) and the edges with negative curvature will expand (the edge weight will increase). This property has found important applications in community detection, as nodes within the same community will get closer to each other, while moving away from nodes belonging to other communities. The Ollivier Ricci Flow has also been used in other contexts, such as to study the fragility of financial markets through the analysis of the correlation between the robustness of a graph and its curvature [6], to study information propagation on a graph and its applications to the over-squashing phenomenon of Graph Neural Networks [7], to study the internet topology [8], and many more.

To give an intuitive example on the idea behind the algorithm, we have implemented the Ollivier Ricci Flow to the *Zachary's Karate Club* dataset [9]. This is a famous social network used broadly in network analysis. It is well-known that this dataset contains two communities. In the image below the weights and curvature are analyzed before and after implementing the Ricci Flow. We are considering the unweighted version of the data, so before implementing the Ricci Flow, all the weights are 1 (lower left diagram) and there is a mixture of curvatures ranging from -2 to 1 (upper left diagram). After the Ricci Flow, all the edge curvatures have converged to 0 (upper right diagram), while the weights now range from 0 to 7 (lower right diagram). Notice that the weight of the edges that initially had a negative curvature has increased after the Ricci Flow (expanded), while the others have decreased (shrunk). The two communities can be clearly identified after the Ricci Flow from the lower right diagram, and correspond to the true values of the data. This is just an intuitive example on some properties of the algorithm, a more in depth analysis will follow in the next sections.

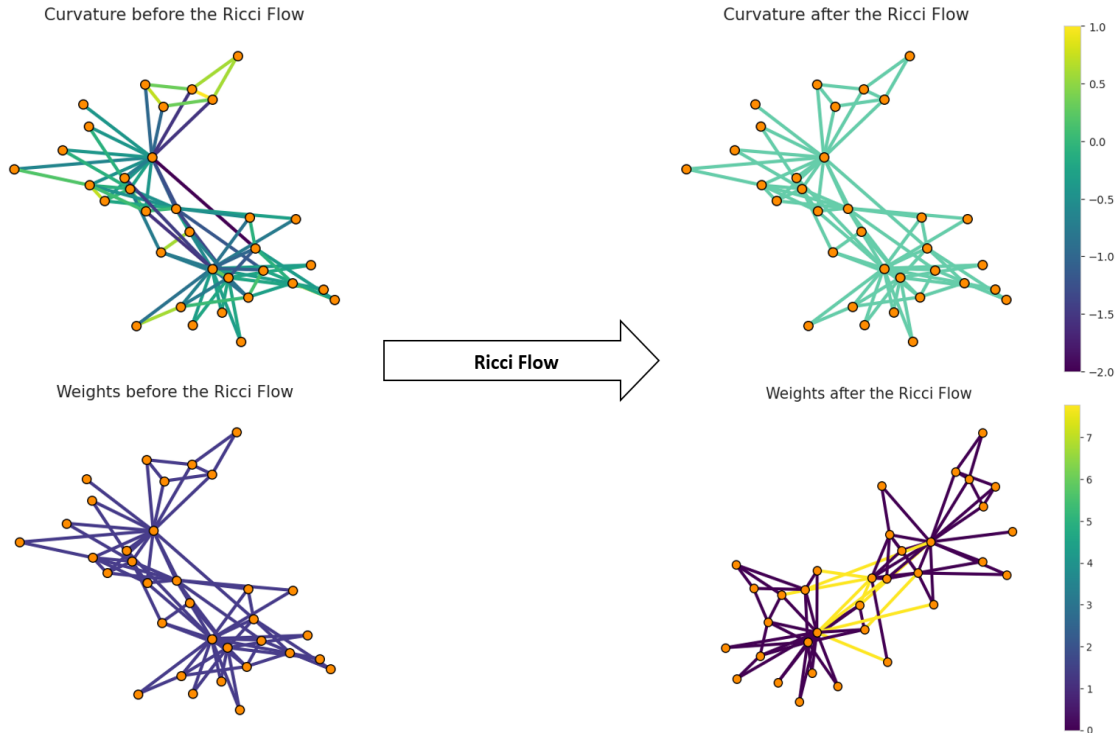


Figure 5: Implementation of the Ricci Flow to the *Zachary's Karate Club* network. The value of the edge weights and curvature can be seen from the colour-bars before (left) and after (right) the Ricci Flow

3.1 Construction

There are different approaches used for implementing the Ricci Flow. In this thesis, we would follow the methodology described in the paper: *Community detection on networks with Ricci flow* [10]. In particular, we followed the Python code that they made available on Github and documented the steps of their computation. The key idea is that at every iteration of the Ricci Flow, the mass distribution is re-defined using the normalized weights from the previous iteration, and the curvature is re-calculated using formula 3. The new weights of every edge j are then updated using $w_j^{t+1} = (1 - k_j^t) \cdot w_j^t$. We then normalize the weights to add up to N and repeat the process. A more detailed explanation of the algorithm is provided below:

Algorithm 1 Mass distribution around a node x

Input: Node x , Edge weights $\mathbf{w}$ of G , idleness α , parameter p

```

1: function Mass Distribution( $x, \mathbf{w}, \alpha, p$ ) ▷ Array of weights of all edges
2: for each vertex  $y$  in  $V(G)$  do
3:   if  $y$  in  $N_x$  then
4:     if  $y=x$  then
5:        $m_x^\alpha(y) = \alpha$ 
6:     else if  $y \neq x$  then
7:        $m_x^\alpha(y) = \frac{(1-\alpha)e^{-w_{xy}^p}}{\sum_{z \in N_x} e^{-w_{zx}^p}}$ 
8:     end if
9:   else if then
10:     $m_x^\alpha(y) = 0$ 
11:   end if
12: end for
13: return  $m_x^\alpha$ 

```

Algorithm 2 Ricci Curvature single edge

Input: Adjacent nodes x and y , Edge weights $\mathbf{w}$ of G , idleness α , parameter p

```

1: function Ricci Curvature( $x, y, \mathbf{w}, \alpha, p$ ) ▷ Curvature of the edge  $xy$ 
2:  $m_x^\alpha = \text{Call Mass Distribution}(x, \mathbf{w}, \alpha, p)$ 
3:  $m_y^\alpha = \text{Call Mass Distribution}(y, \mathbf{w}, \alpha, p)$ 
4:  $k_{xy} = 1 - \frac{W[m_x^\alpha, m_y^\alpha]}{w(x,y)}$  ▷ The Wasserstein distance is calculated using the Sinkhorn library [11].
5: return Ricci Curvature  $xy$ 

```

Algorithm 3 Ricci Flow

Input: Undirected graph G , Number of Iterations $\mathcal{I}$, idleness α , parameter p

```

1: for  $t \leftarrow 1$  to  $\mathcal{I}$  do
2:   for each  $j$  in  $E(G)$  do
3:      $k_j^t = \text{Call Ricci Curvature}(x, y, \mathbf{w}^t, \alpha, p)$  ▷  $xy$  are the nodes adjacent to edge  $j$ 
4:      $w_j^{t+1} = (1 - \epsilon \cdot k_j^t) \cdot w_j^t$  ▷  $\epsilon$  is the step size
5:   end for
6:   for each  $j$  in  $E(G)$  do
7:      $w_j^{t+1} = \frac{w_j^t \times |E(G)|}{\sum_{j \in E(G)} w_j^t}$  ▷ Normalize edge weights used for the next iteration
8:   end for
9: end for
10: return Final weights

```

3.2 Convergence of Ricci Flow

The convergence of the Ricci Flow is of great importance when applying the algorithm to real-world datasets. Even though a number of experimental analyses have been carried out, no theoretical guarantees about its convergence has been obtained yet. This is a particularly difficult task due to the following factors:

1. In general, there is no explicit formula for the Wasserstein distance between masses distributions around adjacent edges and, hence, for the curvature of an edge.
2. The normalization factor implemented at the end of every iteration of the Ricci Flow avoids expansion of the weights to infinity, but makes the analysis significantly more complicated. This is because even though the Wasserstein distance between masses distribution around adjacent nodes may decrease from one iteration to another, the weight of the edge can still increase if the sum of the total Wasserstein distance from all the edges of the graph has also decreased. Therefore, because of the normalization factor, the weight of an edge at the end of one iteration doesn't depend only on the local neighborhood around the edge, but on the global structure of the graph.
3. The shortest path between two nodes can change as the weight of the edges changes, making difficult (or impossible) to track what is the optimal path between two nodes at a given time.

In this section, we will analyze the convergence of the Ricci Flow using a simplified version of the mass distribution around a node x described in Algorithm 1. We will explain how this simplification mitigates some of the difficulties in the analysis outlined above. As stated in Algorithm 1, the mass distribution is dependent of a parameter p which quantifies how much we want to disregard the neighbours that are *far away* (have a large edge weight). Mainly, the mass distribution has the following form:

$$m_x^\alpha(y) = \begin{cases} \alpha & \text{if } y = x \\ \frac{(1-\alpha)e^{-w_{xy}^p}}{\sum_{z \in N_x} e^{-w_{zx}^p}} & \text{if } y \in N_x \\ 0 & \text{Otherwise} \end{cases} \quad (5)$$

As the value of p grows, $\psi(w_{xy}) = e^{-w_{xy}^p}$ becomes steeper around $x=1$. For our analysis, we will approximate this behaviour using the step function below:

$$\phi(w_{xy}) = \begin{cases} 1 & \text{if } w_{xy} \leq 1 \\ 0 & \text{Otherwise} \end{cases} \quad (6)$$

Therefore, the mass distribution around node x that we would be considering is:

$$m_x^\alpha(y) = \begin{cases} \alpha & \text{if } y = x \\ \frac{(1-\alpha)\phi(w_{xy})}{\sum_{z \in N_x} \phi(w_{xz})} & \text{if } y \in N_x \text{ and } \sum_{z \in N_x} \phi(w_{xz}) \neq 0 \\ \frac{(1-\alpha)}{|N_x|} & \text{if } y \in N_x \text{ and } \sum_{z \in N_x} \phi(w_{xz}) = 0 \\ 0 & \text{Otherwise} \end{cases} \quad (7)$$

That is, if the edges around node x all have weight smaller than 1 or all weight larger than 1, we equidistribute the mass. If some edges have weight larger than 1 and some smaller than one, then we equidistribute the mass between the edges with weight smaller than 1 only.

Conceptually, with mass distribution (7), we are completely disregarding the neighbours that have edge weight larger than 1. Thus, this may not be best choice in real-world applications, but is convenient for theoretical analysis.

Let w_j^t be the weight of edge j at iteration t . From Algorithm 3,

$$w_j^{t+1} = (1 - \epsilon \cdot k_j^t) \cdot w_j^t \quad (8)$$

and the curvature of edge j at iteration t , k_j^t , is

$$k_j^t = 1 - \frac{W[m_x^\alpha, m_y^\alpha]}{w_j^t} \quad (9)$$

where x and y are the nodes adjacent to edge j and ϵ is the step size, which we would set equal to 1. Combining expression 8 and 9 together, leads to

$$w_j^{t+1} = W[m_x^\alpha, m_y^\alpha] \quad (10)$$

Let $\mathbf{w}^t$ be an n -dimensional vector, where a $n=|E(G)|$, containing the weights of all edges at iteration t . Now, for a fixed α , we can define a $n \times n$ matrix A whose entry A_{jk} is the mass transported around edge k when computing the optimal Wasserstein distance between the masses distributions adjacent to edge j . Note that we can define the matrix A because the mass distribution described in expression (7) only depends on the edge weight of the neighboring edges being larger or smaller than 1, but not on the value of the weight itself. So the coefficients A_{jk} are not depend on $\mathbf{w}$. Then, we can rewrite the non-normalized weights at iterations $t+1$ as

$$\mathbf{w}^{t+1} = A\mathbf{w}^t$$

If sequential weights have the same optimal transport plan, they will share the same matrix A , so we can express the weights at iteration t as

$$\mathbf{w}^t = A^t \mathbf{w}^0$$

where $\mathbf{w}^0$ are the initial weights associated with A . Realistically, this will only happen in small graphs. We have not proved that this formulation is possible for a general graph, but it certainly is for the small graph that we will be analyzing in the next section. In the case where this formulation is feasible, the step function in expression (7) we applied to the mass distribution has allowed us to write down this problem a system of piecewise linear maps. This has made the problem significantly easier to tackle, as we can now use linear algebra techniques to investigate the convergence. As a future work, it would be interesting to prove if this formulation is correct for a general graph. Assuming the formulation above is feasible for a graph G , we can divide the parameter space into different regions following the below procedure.

1. If the mass distribution around adjacent nodes changes, the optimal transportation plan will definitely change and, hence, the entries of the matrix A too. The mass distribution will change if and only if the weight of an edge changes from smaller than 1 to larger than 1, or viceversa. Therefore, every matrix A has associated a set of conditions which specify if the weight of every edge should be smaller or larger than 1. These set of conditions can be written as the intersection of n half spaces, so every region associated with a matrix A can be viewed as a convex polytope. The sample space Ω will be the union of finitely many convex polytopes. In particular, for a graph with n edges there would be at most 2^n different combinations, so Ω will be the union of at most 2^n convex polytopes. In the diagram below there is a representation of such space.

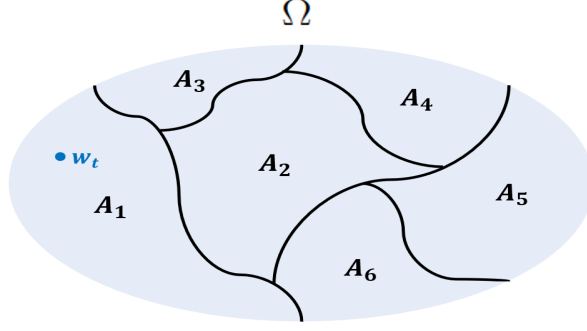


Figure 6: Representation of the sample space Ω , Every region A_i is a convex polytope that determine which edge weights are bigger or smaller than 1

2. Given that $\mathbf{w}^t$ lies inside one of the regions, let's say A_1 , the coefficients of the matrix can still change if the optimal transport plan changes. This can happen, for example, if the shortest path between two nodes has changed with respect to the previous iteration. Hence, every polytope is subdivided by different regions determined by the set of optimal transport plans, as seen below:

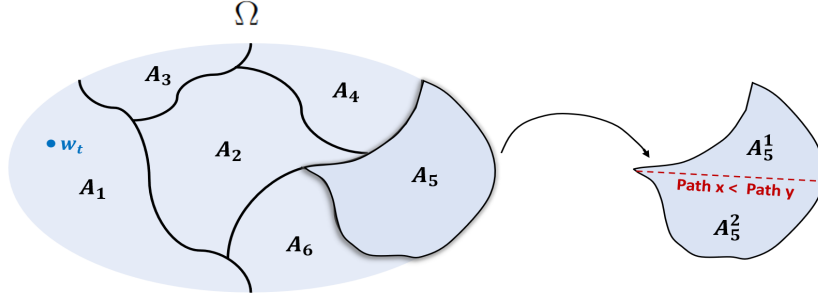


Figure 7: Every region in Ω is subdivided by the different determined by the set of optimal transportation plans

With this construction in mind we are now ready to study how the weights $\mathbf{w}^t$ change throughout the Ricci Flow. To do that, we need to find an answer to the following question: Does there exist a t such that $\mathbf{w}^t = A^t \mathbf{w}^0$ ($\mathbf{w}^0$ being the initial weights associated with A) will remain in the same region A or $\mathbf{w}^t$ will keep *jumping* between regions in Ω ? We will try to find an answer to this question by looking at the direction along the largest eigenvector associated with A . Assuming that A is diagonalizable, we can compute the eigendecomposition of A as below, where D is an $n \times n$ diagonal matrix whose entries are the eigenvalues $\{\lambda_i\}_{i=0}^n$, and P is an $n \times n$ matrix whose columns are the eigenvectors.

$$\mathbf{w}^t = P D^t P^{-1} \mathbf{w}^0$$

Let $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ be the set of normalized eigenvectors. We can then express the weights $\mathbf{w}$ in the basis of $\mathcal{B}$:

$$[\mathbf{w}^t]_{\mathcal{B}} = D^t [\mathbf{w}^0]_{\mathcal{B}}$$

where $[\mathbf{w}^0]_{\mathcal{B}} = \sum_{i=0}^n a_i \mathbf{v}_i$, for some $a_i \in \mathbb{C}$. Then,

$$[\mathbf{w}^t]_{\mathcal{B}} = D^t [\mathbf{w}^0]_{\mathcal{B}} = a_0 \lambda_0^t \mathbf{v}_0 + a_1 \lambda_1^t \mathbf{v}_1 + \dots + a_n \lambda_n^t \mathbf{v}_n$$

Normalizing,

$$[\mathbf{w}_N^t]_{\mathcal{B}} = \frac{a_0 \lambda_0^t \mathbf{v}_0 + a_1 \lambda_1^t \mathbf{v}_1 + \dots + a_n \lambda_n^t \mathbf{v}_n}{\sqrt{a_0^2 \lambda_0^{2t} + a_1^2 \lambda_1^{2t} + \dots + a_n^2 \lambda_n^{2t}}}$$

Let λ_k be the eigenvalues with the largest value in modulus and $\mathbf{v}_k$ its associated eigenvector. Then, as $t \rightarrow \infty$

$$[\mathbf{w}_N^t]_{\mathcal{B}} \rightarrow \mathbf{v}_k$$

If the largest eigenvalue λ_k has multiplicity $1 < m < n$, with associated eigenvectors $\{\mathbf{v}_1, \dots, \mathbf{v}_m\}$, then as $t \rightarrow \infty$,

$$[\mathbf{w}_N^t]_{\mathcal{B}} \rightarrow \frac{a_1 \mathbf{v}_1 + a_2 \mathbf{v}_2 + \dots + a_m \mathbf{v}_m}{\sqrt{a_1^2 + a_2^2 + \dots + a_m^2}} \quad (11)$$

In summary, given $\mathbf{w}^t$ in region A, as $t \rightarrow \infty$ the weights will convergence to the eigenvector with the biggest eigenvalue (or a combination of eigenvectors if the eigenvalue has multiplicity larger than 1). We then just need to change $[\mathbf{w}_N^t]_{\mathcal{B}}$ to the canonical basis using the matrix of eigenvectors P and normalize to add up to n . If $\mathbf{w}^t$ still belongs to the same region A, then there is a possibility that the algorithm has converged. This is still not sufficient to prove convergence because in equation (10) we are not normalizing the weights after every iteration. So even if the eigenvector $\mathbf{v}_k$ lies inside the same polytope as $\mathbf{w}^0$, due to the normalization, some weights may swap from being smaller than 1 to larger than 1, or viceversa, moving to another polytope and changing the matrix A . Nevertheless, this approach is sufficient to prove non-convergence if we identify a bouncing between two regions, as we will describe in the following section.

3.2.1 Non-Convergence of the Ollivier Ricci Flow

So far, there are no examples in literature of non-convergence of the Ollivier Ricci Flow. We believe that there are two scenarios that can cause the weights to diverge; Firstly, the normalization factor can create a bouncing between edges being larger and smaller than 1 and, secondly, a bouncing can occur between different optimal paths joining two nodes. In this section, we provide an example of non-convergence due to the first reason stated.

Theorem 1: Let $G(V,E)$ be an undirected and unweighted graph consisting of two triangles glued together along an edge, as illustrated in Figure 8. The Ollivier Ricci flow with the mass distribution defined in (7) converges for $\frac{1}{3} \leq \alpha < \frac{1}{2}$ and doesn't converge for $\alpha < \frac{1}{3}$.

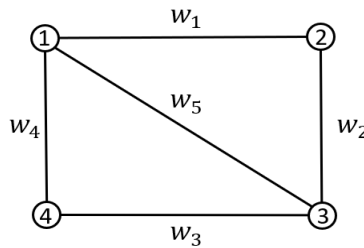


Figure 8: Graph G from Theorem 1

We will experimentally and theoretically prove Theorem 1 in the following sections. The labelling of the nodes and the edges of G illustrated in Figure 8 is the one we will refer to throughout the next sections.

3.2.2 Experimental Analysis

We have used the Python code provided in [10] to investigate the convergence of the Ricci Flow for graph G . The function I have coded to generate the following graphs can be found in Appendix III. The experimental analysis has been very important to get an intuition of how the algorithm

behaves, which has been very useful when constructing the theoretical poof of Theorem 1 in the next section.

In Figure 9, we observe that the Ricci Flow diverges for $\alpha = \frac{1}{4}$ and converges for $\alpha = \frac{1}{3}$ and $\alpha = \frac{1}{2}$. In addition, for $\alpha = \frac{1}{3}$ the weight of the exterior edges converges to $\frac{15}{13}$ and the weight of the interior edge converges to $\frac{5}{13}$ in just one iteration, and for $\alpha = \frac{1}{2}$ the weight of the exterior edges converges to $\frac{5}{4}$ and the weight of the interior edge converges to 0. We would prove all these results theoretically in the next section.

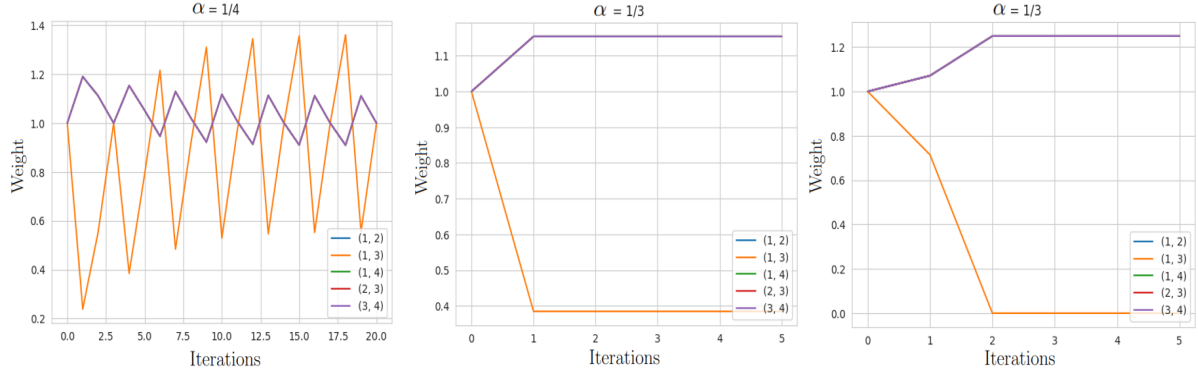


Figure 9: Graphs of the edge weights in every iteration of the Ricci Flow for $\alpha = \frac{1}{4}, \alpha = \frac{1}{3}, \alpha = \frac{1}{2}$

We have also plotted the change in the weights of the exterior edges along the Ricci Flow for different values of α in a 3D plot as seen in Figure 10 below. It can be clearly seen how the weights bounce for $\alpha < \frac{1}{3}$ and converge otherwise.

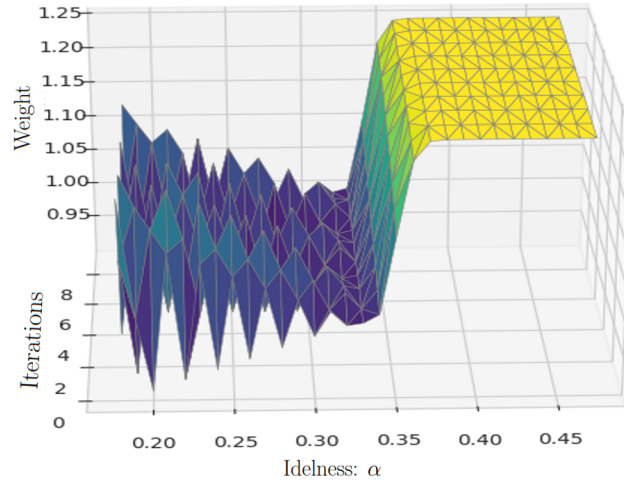


Figure 10: 3D plot of the weights of the exterior edge along iterations of the Ricci Flow for different values of α

3.2.3 Proof of Theorem 1

We will proof Theorem 1 using the approach that we described in Section 3.2. This graph only has two types of edges: the exterior and the interior ones. Therefore, we can divide the sample space Ω on three convex polytopes A_1 , A_2 and A_3 , defined as below:

$$\mathbf{w}^t \in A_1 \text{ if } \mathbf{w}_1 = \mathbf{w}_2 = \mathbf{w}_3 = \mathbf{w}_4 = \mathbf{w}_5 = 1$$

$$\mathbf{w}^t \in A_2 \text{ if } \mathbf{w}_5 \leq 1 \text{ and } \mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3, \mathbf{w}_4 > 1$$

$$\mathbf{w}^t \in A_3 \text{ if } \mathbf{w}_5 > 1 \text{ and } \mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3, \mathbf{w}_4 \leq 1$$

Each one of these regions is subdivided into smaller sub-regions determined by the set of optimal transport plans, as we explained in Section 3.2. In particular, for a fixed α , when $\mathbf{w}_t \in A_3$ there are two possible optimal paths from node 1 to node 3: Through the diagonal (edge 5) or through the exterior edges (edge 1 and 2, or edge 3 and 4), as can be seen in Figure 11 below. We will discuss these subregions in more detail in the next pages.

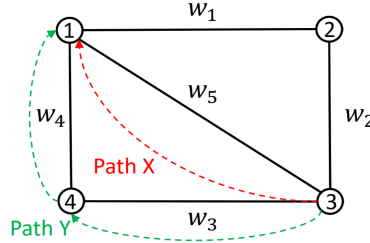


Figure 11: Graph G with labelling of nodes and edges. There are two possible optimal paths joining node 1 and node 2: Path X and Path Y

We call the first path *Path X* and the second one *Path Y*. Therefore, putting these together we get to the following representation of the sample space Ω for G:

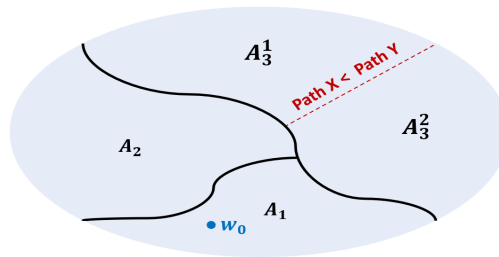
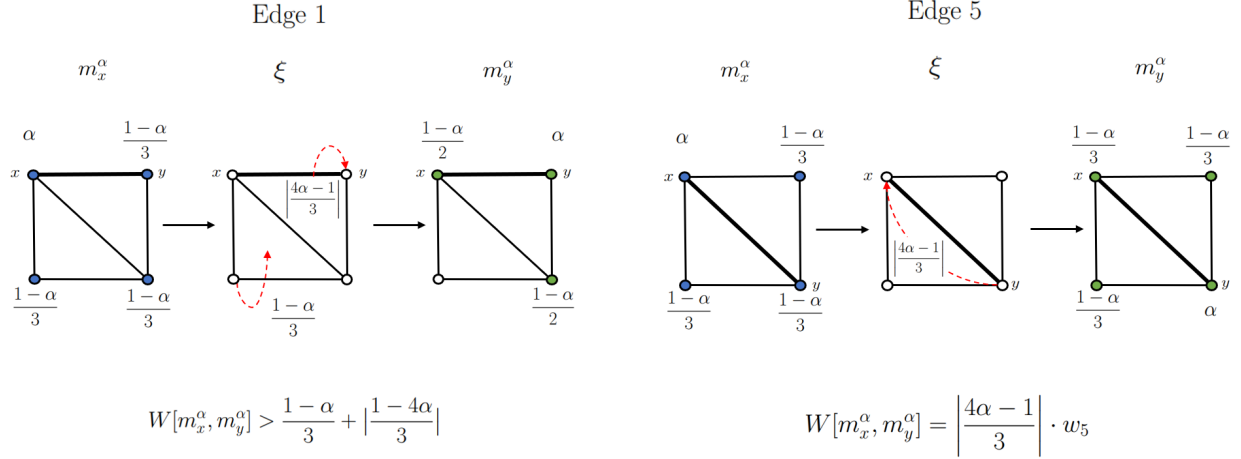


Figure 12: Representation of the sample space Ω for the graph G

As G is an unweighted graph, $\mathbf{w}^0$ initially lies in region A_1 . The next step is to calculate the Wasserstein distance between adjacent nodes. For edges 1,2,3,4 (external edges), there are 3 optimal transportation plans depending on the value of α . In particular, for $\alpha < \frac{1}{7}$, $\frac{1}{7} < \alpha < \frac{1}{2}$ and $\alpha > \frac{1}{2}$. To avoid considering these cases separately, we will find a common lower bound for the Wasserstein distance and we will then prove that this bound is enough to make $w_5^1 < 1$ and $w_1^1 = w_2^1 = w_3^1 = w_4^1 > 1$. The figure and table below outlines the steps used to obtain the lower bound in the Wasserstein distance for edges 1,2,3,4 and the step used to obtain the optimal transportation plan in edge 5.



Optimal transport plan edges 1,2,3,4

1. As m_y^α does not contain any mass at node 4, we will have to move $\frac{1-\alpha}{3}$ mass from node 4 in m_x^α . Depending on the value of α , the optimal transport plan will move the mass to node 1 or to node 3.
 2. Regardless of the outcome of the last step, the optimal transport plan will have to move at least $|\frac{1-\alpha}{3} - \alpha| = |\frac{1-4\alpha}{3}|$ mass to node 2 (the difference between the two masses distributions at node 2).
 3. Then, $W[m_x^\alpha, m_y^\alpha] > \frac{1-\alpha}{3} + \left| \frac{1-4\alpha}{3} \right|$
-

Optimal transport plan edge 5

1. We need to rearrange the remaining mass between node 1 and 3. As the weight of all the edges is the same, the optimal transport plan will move the mass along edge 5. Notice that the optimal transport plan is the same for all values of α .
 2. Then, $W[m_x^\alpha, m_y^\alpha] = \left| \frac{1-4\alpha}{3} \right|$
-

To obtain the values of the weight after the first iteration, we need to normalize the Wasserstein distances obtained above. One can check that for edge 5,

$$w_5^1 < \frac{5 \left| \frac{1-4\alpha}{3} \right|}{\left| \frac{1-4\alpha}{3} \right| + 4 \left(\frac{1-\alpha}{3} + \left| \frac{1-\alpha}{3} \right| \right)} = \frac{5 \left| \frac{1-4\alpha}{3} \right|}{5 \left| \frac{1-4\alpha}{3} \right| + 4 \left(\frac{1-\alpha}{3} \right)} < 1, \forall \alpha$$

And, by consequence, we will have that the weight of all the other edges will be larger than 1. Hence, we have moved to the A_2 region. In the diagram below one can see the masses distributions for edge 1 and edge 5 respectively in A_2 . As a reminder, here we are using the mass distribution defined in expression (7), so we only put mass along the edges with weight smaller than 1 (or equidistribute if all the adjacent edges have weight larger than 1). That is the reason why in the masses distribution in Edge 5 we don't put mass in node 2 or 4. We claim that for $\alpha < \frac{1}{2}$ (the case that we are considering in Theorem 1), there is one optimal transport plan in A_2 . The table below outlines the reason behind it.

Case 1: $\alpha < \frac{1}{3}$. The eigenvector associated with the largest eigenvalue is $(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}, 1)^T$. Changing to the canonical basis using the matrix of eigenvectors above, we get that the weights converge to $(0, 0, 0, 0, 1)^T$. This doesn't belong to the region A_2 anymore. As $w_5 > w_1, w_2, w_3, w_4$, it belongs to A_3 . Nevertheless, we still need to determine if the weights will be in the A_3^1 or A_3^2 region. That is, whether the optimal path is *Path X* or *Path Y* from Figure 11.

Lemma 2: Given that the weights of graph G are such that $w_5 < w_1, w_2, w_3, w_4$, the resulting weights obtained by applying A_2 are such that the *Path X* is always the optimal path between node 1 and 3.

Proof: To prove that *Path X* is the optimal path between node 1 and 3, we need to prove that the resulting weights after applying A_2 are such that $w_5^t < w_1^t + w_2^t$. Due to normalization we know that at the end of the previous iteration $w_1^t + w_2^t + w_3^t + w_4^t + w_5^t = 5$ and due to the symmetry of the graph we have that $w_1^t = w_2^t = w_3^t = w_4^t$. Putting these two results together, we get that $w_5 + 4w_1 = 5$. So, $w_5^t < w_1^t + w_2^t \rightarrow w_5^t < 2w_1 \rightarrow w_5^t < \frac{5}{3}$. Hence, to prove that $w_5^t < w_1^t + w_2^t$ we need to prove that $w_5^t < \frac{5}{3}$. We don't know the exact values of the weight from the previous iteration, so we would have to work with general values instead. Let $\mathbf{w}^{t-1} = (1 + \delta, 1 + \delta, 1 + \delta, 1 + \delta, 1 - 4\delta)^T$ where $0 < \delta < \frac{1}{4}$. (Note that as $\mathbf{w}^{t-1} \in A_2, w_5 < 1$ and $w_1, w_2, w_3, w_4 > 1$). We need to prove that for all $0 < \delta < \frac{1}{4}$ and $0 < \alpha < \frac{1}{3}$, the resulting weights after applying A_2 to $\mathbf{w}^{t-1}$ are such that $w_5 < \frac{5}{3}$.

$$\begin{pmatrix} 0 & \alpha & 0 & 0 & \frac{1-3\alpha}{2} \\ 0 & 0 & \alpha & 0 & \frac{1-3\alpha}{2} \\ 0 & 0 & 0 & \alpha & \frac{1-3\alpha}{2} \\ \alpha & 0 & 0 & 0 & \frac{1-3\alpha}{2} \\ 0 & 0 & 0 & 0 & 1-2\alpha \end{pmatrix} \begin{pmatrix} 1+\delta \\ 1+\delta \\ 1+\delta \\ 1+\delta \\ 1-4\delta \end{pmatrix} = \begin{pmatrix} \alpha(\delta+1) + \frac{1-3\alpha}{2}(1-4\delta) \\ \alpha(\delta+1) + \frac{1-3\alpha}{2}(1-4\delta) \\ \alpha(\delta+1) + \frac{1-3\alpha}{2}(1-4\delta) \\ \alpha(\delta+1) + \frac{1-3\alpha}{2}(1-4\delta) \\ (1-2\alpha)(1-4\delta) \end{pmatrix}$$

Normalizing, we get that $w_5^t = 5 \frac{(1-2\alpha)(1-4\delta)}{\alpha(9\delta-4) - 3(\delta-1)}$. Plotting this inequality on Python (Figure 18 below), we indeed get that $w_5^t = 5 \frac{(1-2\alpha)(1-4\delta)}{\alpha(9\delta-4) - 3(\delta-1)} < \frac{5}{3}, \forall \alpha \in (0, \frac{1}{3}), \forall \delta \in (0, \frac{1}{4})$, as required.

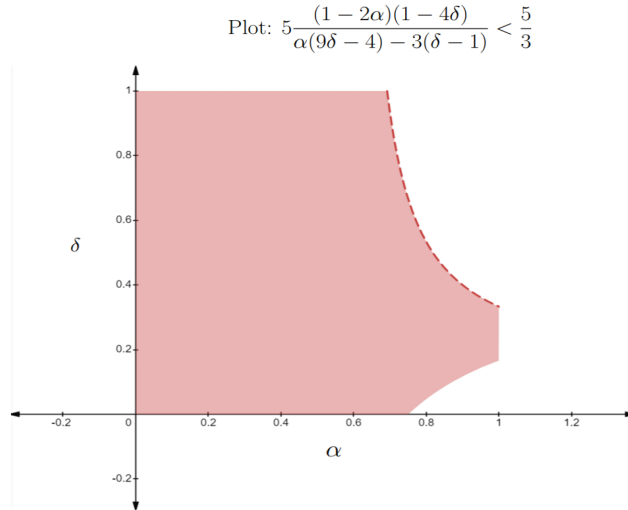
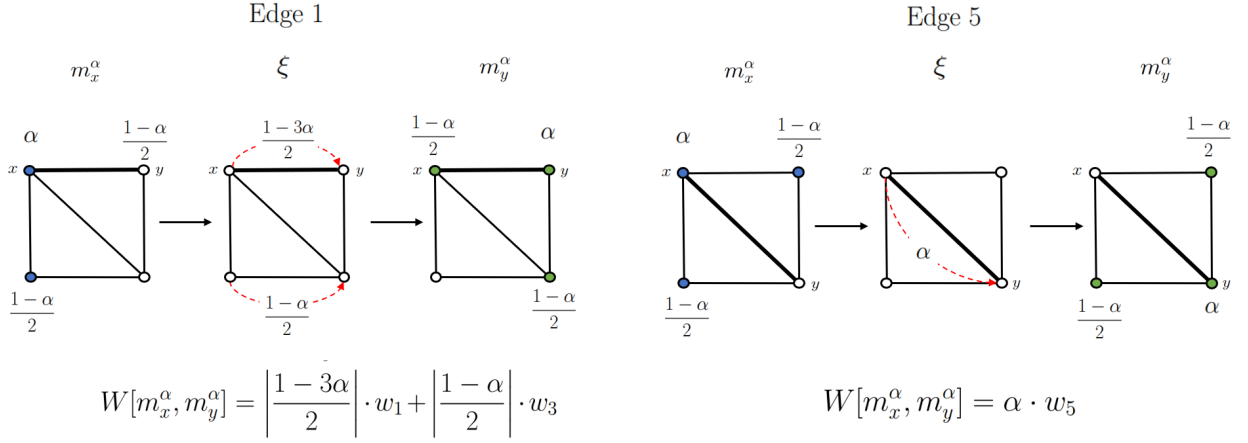


Figure 13: Region satisfying the inequality

In conclusion, $\mathbf{w}^t$ has moved from the polytope A_2 to the polytope A_3^1 . Therefore, we need to compute the new matrix A_3^1 following the same procedure as before. Now, $w_5 > 1$ and $w_1, w_2, w_3, w_4 < 1$, so the mass distribution is as follows:



On the table below there is a justification of the optimal transport plan.

Optimal transport plan edges 1,2,3,4	Optimal transport plan edge 5
- Starting at the distribution around x, we need to move all the mass from node 4. As node 3 needs the same mass contained in node 4, the optimal transport plan is to move the mass from node 4 to node 3 along edge 3. - We then move the extra mass from node 2 to node 1 along edge 1.	To move from m_x^α to m_y^α we only need to redistribute the mass in node 1 and 3. As we have proved in Lemma 2 that Path X is the optimal path, we obtain that the optimal transportation plan is just to move α along edge 5.

Using these transportation plans, we can construct the matrix A_3^1 as follows:

$$\begin{pmatrix} w_1^t \\ w_2^t \\ w_3^t \\ w_4^t \\ w_5^t \end{pmatrix} = \begin{pmatrix} \frac{1-3\alpha}{2} & 0 & \frac{1-\alpha}{2} & 0 & 0 \\ 0 & \frac{1-3\alpha}{2} & 0 & \frac{1-\alpha}{2} & 0 \\ \frac{1-\alpha}{2} & 0 & \frac{1-3\alpha}{2} & 0 & 0 \\ 0 & \frac{1-\alpha}{2} & 0 & \frac{1-3\alpha}{2} & 0 \\ 0 & 0 & 0 & 0 & \alpha \end{pmatrix}^t \begin{pmatrix} w_1^0 \\ w_2^0 \\ w_3^0 \\ w_4^0 \\ w_5^0 \end{pmatrix}$$

Its eigendecomposition is

$$\begin{pmatrix} 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}^{-1} \begin{pmatrix} \alpha & 0 & 0 & 0 & 0 \\ 0 & -\alpha & 0 & 0 & 0 \\ 0 & 0 & -\alpha & 0 & 0 \\ 0 & 0 & 0 & 1-\alpha & 0 \\ 0 & 0 & 0 & 0 & 1-\alpha \end{pmatrix} \begin{pmatrix} 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

As $\alpha < \frac{1}{3}$, the biggest eigenvalue is $1 - \alpha$ with associated eigenvectors $\left\{ (1, 0, 1, 0, 0)^T, (0, 1, 0, 1, 0)^T \right\}$. To find the weights where these eigenvectors converge to, we need to find the coefficients a_1 and a_2 from the expression 11. We don't know the exact values of $[w^0]_{\mathcal{B}}$ (as we don't the exact values of the weights after the 1st iteration), but due to the symmetry of the graph $w_1^0 = w_2^0 = w_3^0 = w_4^0$, hence $a_1 = a_2$, and $[\mathbf{w}]_{\mathcal{B}}$ converges to $(0, 0, 0, 1, 1)^T$. Changing to the canonical basis and normalizing, we get that weights converge to $(\frac{5}{4}, \frac{5}{4}, \frac{5}{4}, \frac{5}{4}, 0)^T$. This doesn't belong to the polytope A_3^1 anymore as clearly $w_5 < w_1, w_2, w_3, w_4$. In fact, $\mathbf{w}_t$ is on the region A_2 again.

In conclusion, for $\alpha < \frac{1}{3}$, $\mathbf{w}_t$ will keep bouncing between A_2 and A_3^1 and the Ollivier Ricci Flow will not converge.

Case 2: $\frac{1}{3} < \alpha < \frac{1}{2}$ The eigenvectors associated with the largest eigenvalue $|\alpha|$ are $\left\{ (-1, 1, -1, 1, 0)^T, (1, 1, 1, 1, 0)^T, (i, -1, -i, 1, 0)^T, (-i, -1, i, 1, 0)^T \right\}$. As before, we need to find the coefficients a_1, a_2, a_3, a_4 respectively from expression 11. By the symmetry of the graph ($w_1^t = w_2^t = w_3^t = w_4^t$), we get that $a_1, a_3, a_4 = 0$, and $a_2 = \frac{2w_1^0}{w_5^0}$, where w_i^0 are the initial weights associated with polytope A_2 . Hence, the weights $[\mathbf{w}^t]_{\mathcal{B}}$ will converge to $(0, \frac{2w_1^0}{w_5^0}, 0, 0, 0)^T$. Changing to the canonical basis and normalizing, we get that the weights converge to $(\frac{5}{4}, \frac{5}{4}, \frac{5}{4}, \frac{5}{4}, 0)^T$. This is still in the A_2 polytope, but this is not enough to prove convergence as we have to make sure that the contribution of the other eigenvectors on the first iterations is not enough to make the weight $\mathbf{w}_t$ change polytope due to normalization. We will prove that this is not the case by manipulating the inequalities associated to the matrix A_2 . Assume that at iteration t the normalized weights $\mathbf{w}_t$ satisfy that $\mathbf{w}_t \in A_2$. This leads to the following set of inequalities:

$$\begin{cases} 4w_5^t < w_1^t + w_2^t + w_3^t + w_4^t \end{cases} \quad (13)$$

$$\begin{cases} 4w_1^t > w_2^t + w_3^t + w_4^t + w_5^t \end{cases} \quad (14)$$

$$\begin{cases} 4w_2^t > w_1^t + w_3^t + w_4^t + w_5^t \end{cases} \quad (15)$$

$$\begin{cases} 4w_3^t > w_1^t + w_2^t + w_4^t + w_5^t \end{cases} \quad (16)$$

$$\begin{cases} 4w_4^t > w_1^t + w_2^t + w_3^t + w_5^t \end{cases} \quad (17)$$

We have to prove that those inequalities are still satisfied throughout the iterations. We will start by analyzing inequality 13. We want to prove that

$$4w_5^{t+1} < w_1^{t+1} + w_2^{t+1} + w_3^{t+1} + w_4^{t+1} \quad (18)$$

We can re-write the weights at the iteration $t+1$ in terms of the weights at iteration t by applying the matrix A_2 to $\mathbf{w}_t$. Substituting the result into inequality 18 we get:

$$4|1 - 2\alpha|w_5^t < \alpha(w_1^t + w_2^t + w_3^t + w_4^t) + 2|1 - 3\alpha|w_5^t$$

Rearranging, we get that

$$4\left(|1 - 2\alpha| - \frac{|1 - 3\alpha|}{2}\right)w_5^t < \alpha(w_1^t + w_2^t + w_3^t + w_4^t)$$

We can now multiply both sides of the first inequality from 13 by $|1 - 2\alpha| - \frac{|1 - 3\alpha|}{2}$. Noticing that $|1 - 2\alpha| - \frac{|1 - 3\alpha|}{2} < \alpha$ for all $\alpha > \frac{1}{3}$ we get that

$$4\left(|1 - 2\alpha| - \frac{|1 - 3\alpha|}{2}\right)w_5^t < \left(|1 - 2\alpha| - \frac{|1 - 3\alpha|}{2}\right)(w_1^t + w_2^t + w_3^t + w_4^t) \leq \alpha(w_1^t + w_2^t + w_3^t + w_4^t), \quad \forall \alpha > \frac{1}{3}$$

Therefore, we have proved that inequality 18 is satisfied on every iteration. The other inequalities can be proven analogously, the procedure for inequality 14 is outlined below. We want to proof that:

$$4w_1^{t+1} > w_2^{t+1} + w_3^{t+1} + w_4^{t+1} + w_5^{t+1}$$

Applying the matrix A_2 and substituting the result into the equation, we get that

$$4\left(\alpha \cdot w_2^t + \left|\frac{1-3\alpha}{2}\right|w_5^t\right) > \alpha(w_1^t + w_3^t + w_4^t) + 2|1-3\alpha|w_5^t + |1-2\alpha|w_5^t$$

Rearranging,

$$\alpha \cdot w_1^t < \alpha(4w_2^t - w_3^t - w_4^t) - |1-2\alpha|w_5^t$$

One can check that $1-2\alpha < \alpha \ \forall \alpha > \frac{1}{3}$. Using the fact that $w_1^t = w_2^t = w_3^t = w_4^t > w_5^t$, we conclude that inequality 18 is satisfied in every iteration.

In conclusion, for $\alpha > \frac{1}{3}$, the weights after the Ricci Flow will converge to $(\frac{5}{4}, \frac{5}{4}, \frac{5}{4}, \frac{5}{4}, 0)^T$. This value coincides with the one obtained experimentally in Section 3.2.2.

Case 3: $\alpha = \frac{1}{3}$ When $\alpha = \frac{1}{3}$, all the eigenvalues have the same magnitude. Let $[\mathbf{w}^0]_{\mathcal{B}} = \sum_{i=0}^n a_i \mathbf{v}_i$ be the initial weights associated to A_2 on the basis of eigenvectors. As $t \rightarrow \infty$, $[\mathbf{w}^0]_{\mathcal{B}}$ will converge to:

$$[\mathbf{w}_N^t]_{\mathcal{B}} \rightarrow \frac{a_1 \mathbf{v}_1 + a_2 \mathbf{v}_2 + \dots + a_n \mathbf{v}_n}{\sqrt{a_1^2 + a_2^2 + \dots + a_n^2}} = \frac{[\mathbf{w}^0]_{\mathcal{B}}}{\sqrt{a_1^2 + a_2^2 + \dots + a_n^2}}$$

Which is just the normalized initial weights. Hence, the weights will remain constant through every iteration. The initial weights in A_2 , $\mathbf{w}^0$, are the weights after the first iteration in A_1 (as the weights move from A_1 to A_2 in just one iteration). So we just need to study the final weights in A_1 to find the exact values where the weights converge to. We will not calculate this value in the dissertation because the procedure is long and analogous to the techniques already used throughout the proof. Broadly speaking, we will have to construct the 3 versions of the matrix A_1 depending on the value of α (as $\alpha < \frac{1}{7}$, $\frac{1}{7} < \alpha < \frac{1}{2}$ and $\alpha > \frac{1}{2}$ have different optimal transport plans). We then just need to apply A_1 to the initial weights and normalize.

In conclusion, for $\alpha = \frac{1}{3}$ the edge weights will converge after 1 iteration. This is consistent with the experimental results, as in the middle graph of Figure 9 one can see that the weights remain constant after the first iteration.

To summarize our analysis, we get that the weights after Ricci Flow the weights will converge/-diverge as follows:

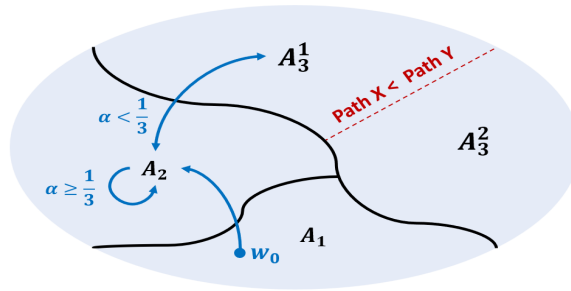


Figure 14: Summary of the convergence of the Ricci Flow for G

These theoretical results are consistent with the experimental results obtained in section 3.2.2.

3.2.4 Interesting points and future work on the convergence of the Ricci Flow

Along the experimental and theoretical analysis we have come across some points that would be interesting to investigate in a future work.

1. Throughout out the theoretical analysis we have noticed that the weights only have a direction towards eigenvectors associated with real and positive eigenvalues. Negative and complex eigenvalues could create another type of bouncing on the edge weights that could prevent convergence. We believe that the there is no feasible optimal transport plan that will give rise to this situation, but it would be interesting to understand theoretically the reason why.
2. In this dissertation, we have only proved the non-convergence of the Ricci Flow for the specific graph G shown in Figure 8. Nevertheless, this is a behaviour that we have seen in other graphs containing a high number of triangles. In Figure 15 there are some examples of non-convergence of the Ricci Flow that we came across during the experimental analysis. An interesting future work will be to study if it is possible to construct an arbitrarily large graph exhibiting this phenomena.

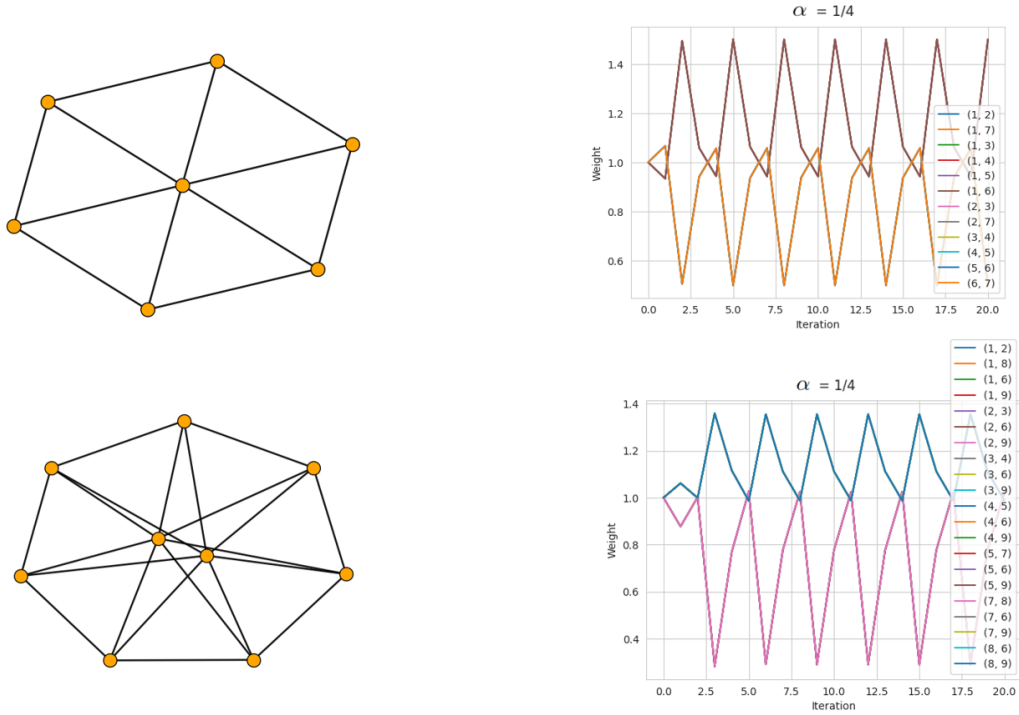


Figure 15: Other examples of non-convergence of Ollivier Ricci Flow

3. We have only proved non-convergence when using the mass distribution defined in 7, which is not the one used for practical purposes as we discussed earlier. Experimentally, we were able to check that using the continuous mass distribution from Algorithm 1, the Ricci Flow applied to G doesn't converge for $p > 5$. This is because for large values of p , the continuous mass distribution defined in Algorithm 1 is *similar enough* to the step function we have been working on (7). It could be interesting to extend our proof for the continuous version of the mass distribution. Our proof could be adapted by adding a small portion of mass δ along the edges with negative curvature and analyzing the convergence using the same techniques. This approach will only work as long as we can express the problem as a system of piecewise linear maps, which as we already discussed, we were not able to prove for the general case.

4 Robustness of Ollivier Ricci Curvature and Ricci Flow

In order to be able to use the Ollivier Ricci Curvature and Ollivier Ricci Flow in real-world applications it is essential for the algorithms to be robust to noise in the input data. To study the robustness of these algorithms we will apply Gaussian noise to the weights of an unweighted graph and study how sensible the algorithms are to those changes. We decided to use Gaussian noise as this represents small random fluctuations in the data that may arise in real-world datasets. In this case, we will use a truncated version of the normal distribution as we don't want the weights to be negative, nor to be larger than the number of edges in the graph. In particular, the noise will follow the below normal distribution with mean 0 and standard deviation σ .

$$\text{Noise}(\sigma) \sim -0.9 \leq \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}(\frac{x}{\sigma})^2} \leq |E(G)|$$

In order to evaluate the robustness of the algorithms, we will express the problem in terms of a classification problem and look at the accuracy. That is, when working with the Ollivier Ricci Curvature, we will first compute the curvature of the edges without the noise. The edges with positive curvature will belong to one group (call it 0) and the edges with negative curvature will belong to the other group (call it 1). We will then apply the noise to the input weights of the graph and look how many edges have been classified correctly. We will repeat this procedure for different values of standard deviation σ . Our intuition is that as the standard deviation increases, the accuracy will decrease. For the Ollivier Ricci Flow, the methodology is the same, but we define one group as being the set of edges with weight smaller than 1 after the implementation of the Ricci Flow (call it 0) and the other group as the set of edges with weight larger or equal than 1 after the implementation of Ricci Flow (call it 1). Finally, the accuracy of the algorithm is defined as follows:

$$\text{Accuracy} = \frac{\text{Number of correct classifications}}{\text{Total number of classifications}}$$

Notice that this is a simplification of the problem, as we are not considering *how much* has the weight of a noisy edge changed with the Ricci Flow compared to its non noisy counterpart. We are only analyzing if the general behaviour of the algorithms (positive/negative curvature and shrunked/expanded edges) is the same.

The dataset that we have used to run our experiments is a random generated network from Python following a Watts–Strogatz model. This model generates graphs with small-world properties. These are types of graphs with small clustering coefficients and that have been found to represent well the dynamics of social networks [12]. The graph we have used contains 200 nodes and 1000 edges. We are not working with larger graphs due to the computational costs of the Ricci Flow.

In the diagram below illustrates the confusion matrix for the Ricci Curvature (right) and Ricci Flow (left) when a Gaussian noise with standard deviation 4 is applied to the initial weights. The algorithm classified most of the edges with initial positive curvature in the wrong group, indicating that these could be the more unstable edges. In the other hand, the Ricci Flow managed to classify most of the edges that initially shrunked (weight smaller than 1) correctly. Overall, the confusion matrices indicate that the Ricci Flow has managed to classify most of the edges in the correct group, while the Ricci Curvature seems to be more sensible to noisy data.

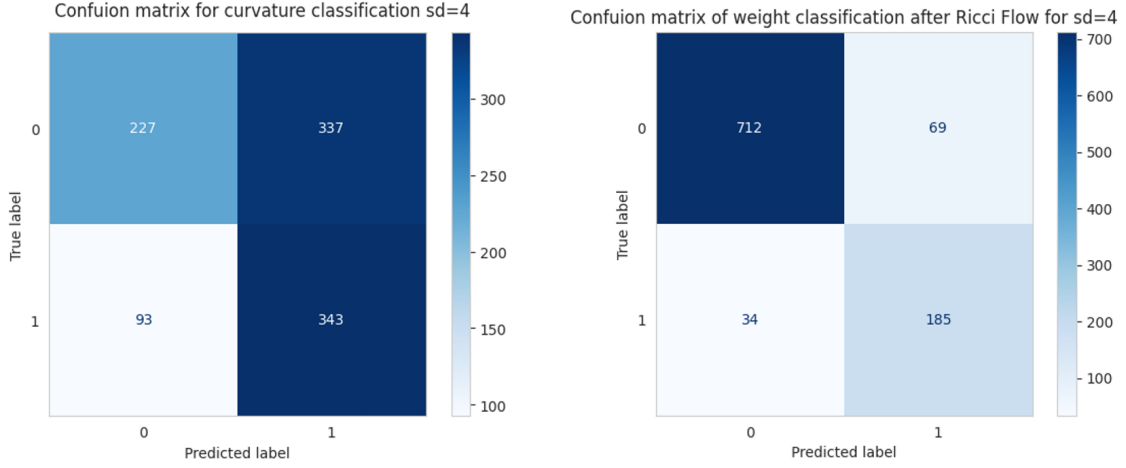


Figure 16: Accuracy in terms of the standard deviation for the Ricci Curvature and Ricci Flow

We then studied how the accuracy of the classification changed as we increase the value of the standard deviation σ . For our experiments, we have used the code outlined in Section 3.1. We have set the parameters $\alpha = 0.2$ and $p = 1$. The results can be seen in Figure 17.

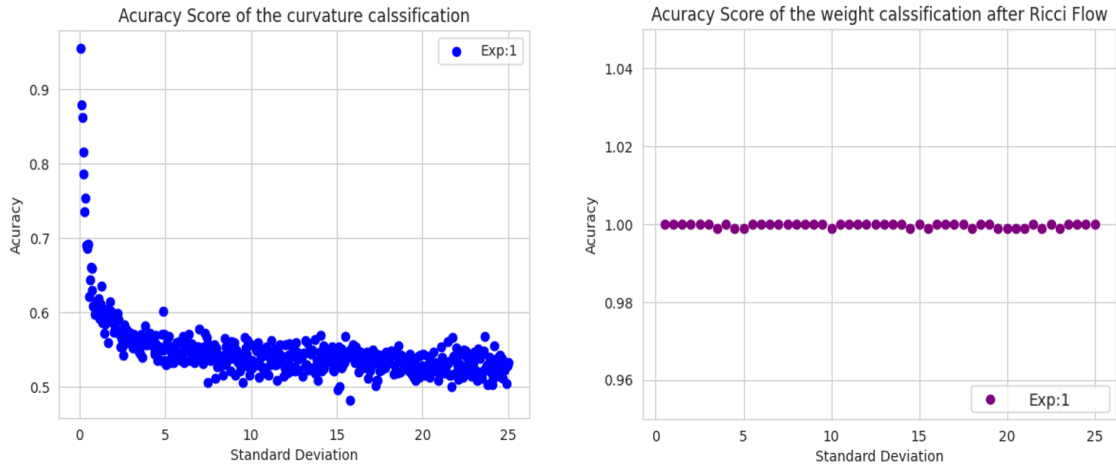


Figure 17: Accuracy in terms of the standard deviation for the Ricci Curvature and Ricci Flow

The accuracy of the Ricci Curvature quickly decreased as the value of the standard deviation increases, while the accuracy of the Ricci Flow is maintained consistently very close to 1. Notice that the accuracy of the curvature tends to 0.5. This is because we are only working with two groups (positive/negative curved edges), so for large values of σ the algorithm is obtaining the same accuracy as if it was guessing the values at random. It is interesting that even though the Ricci Flow is working with noisy values of curvature, it still manages to recover high values of accuracy.

In conclusion, the Ricci Flow seems to be a robust algorithm to noisy data, classifying most of the edges correctly even for large values of the standard deviation. In the other hand, the Ricci curvature appears to be a lot more sensible to changes in the input data. This should be considered when working with these notion of real-world applications.

5 Conclusion

In this report we studied the Ollivier Ricci Curvature and the Ollivier Ricci Flow.

We first studied the definition and properties of the Ollivier Ricci Curvature, and its connection to the Ricci Curvature defined in a manifold. We were able to understand how this notion give us information about the structure of a graph and how can this be used in network analysis applications. We were able to experimentally find regular graphs with constant curvature up to and including 11 vertices using random generator models in Python.

We then studied the Ollivier Ricci Flow. We were able to understand the algorithm used in the research paper *Community detection on networks with Ricci flow* [10] and explore potential applications to community detection. We then proved experimentally and theoretically that the Ollivier Ricci Flow doesn't converge by giving a counterexample. Finally, we implemented Python simulations to understand how robust are those notions to noisy dataset, a fundamental property to be understood for real-world applications. Our results showed that the Ricci Flow appears to be a robust algorithm, even for large value of noise, while the curvature seems to be more sensible to changes in the input data. A more in depth analysis should be carried out with different types of network and different parameters of the Ricci Flow to understand better this behaviour.

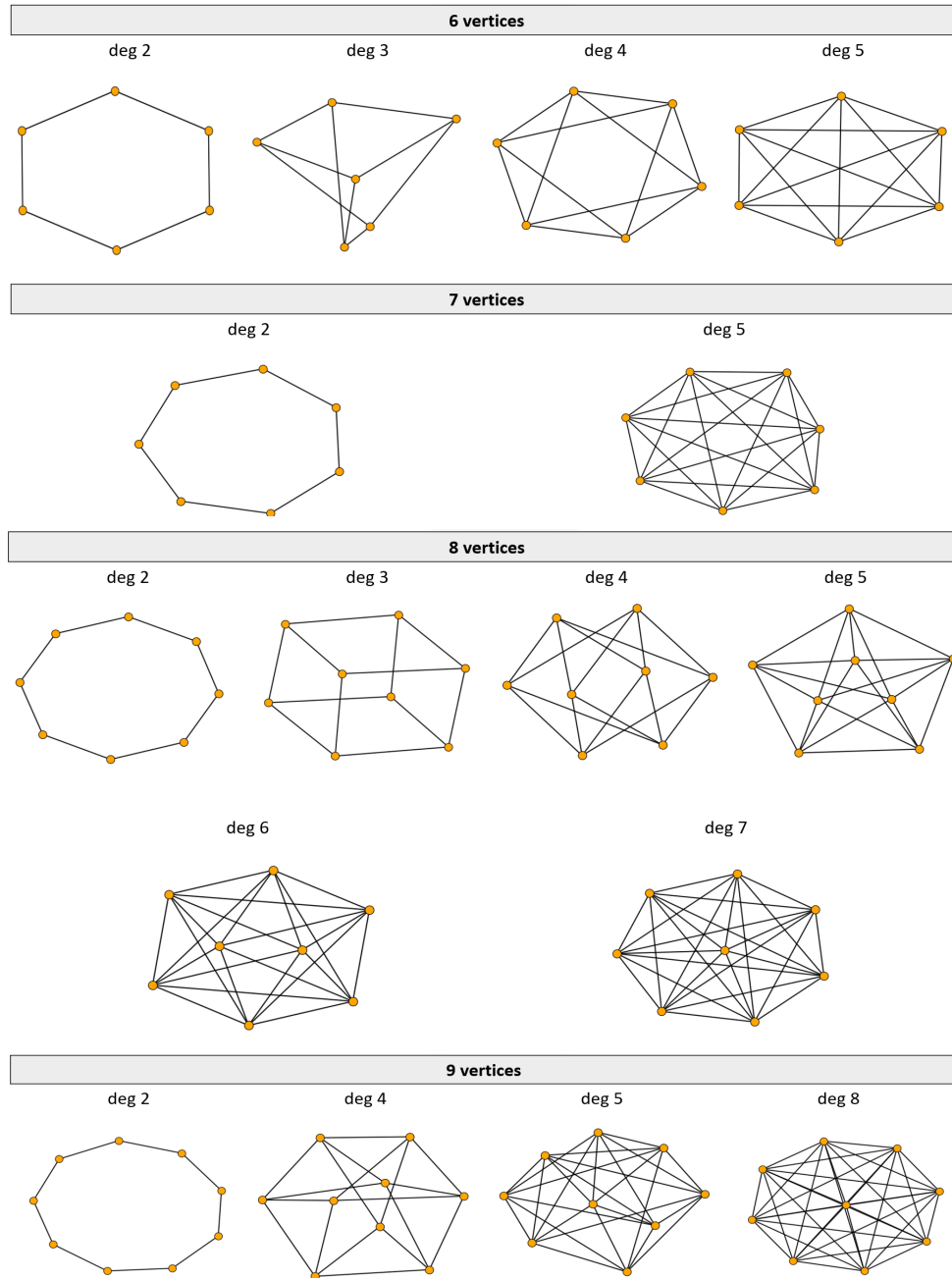
This project also gave rise to numerous future lines of research that we described in Section 3.2.4.

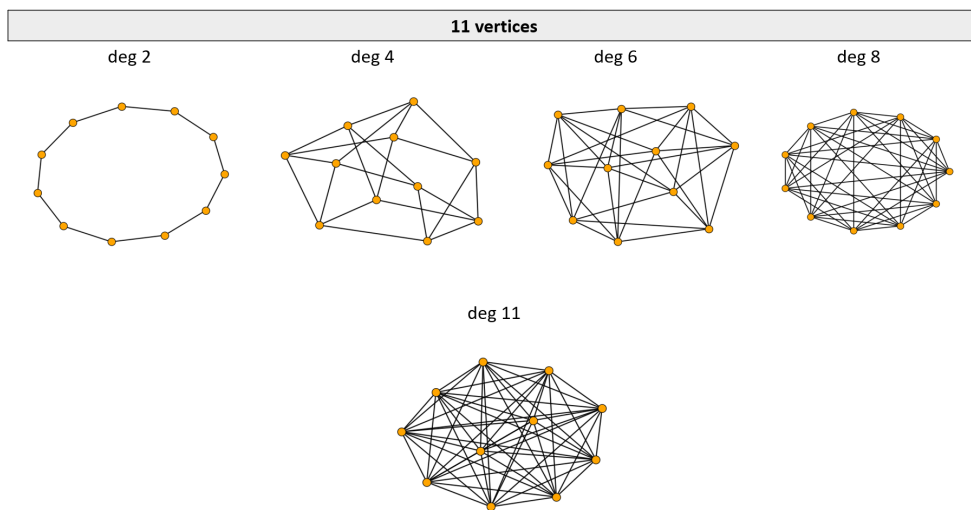
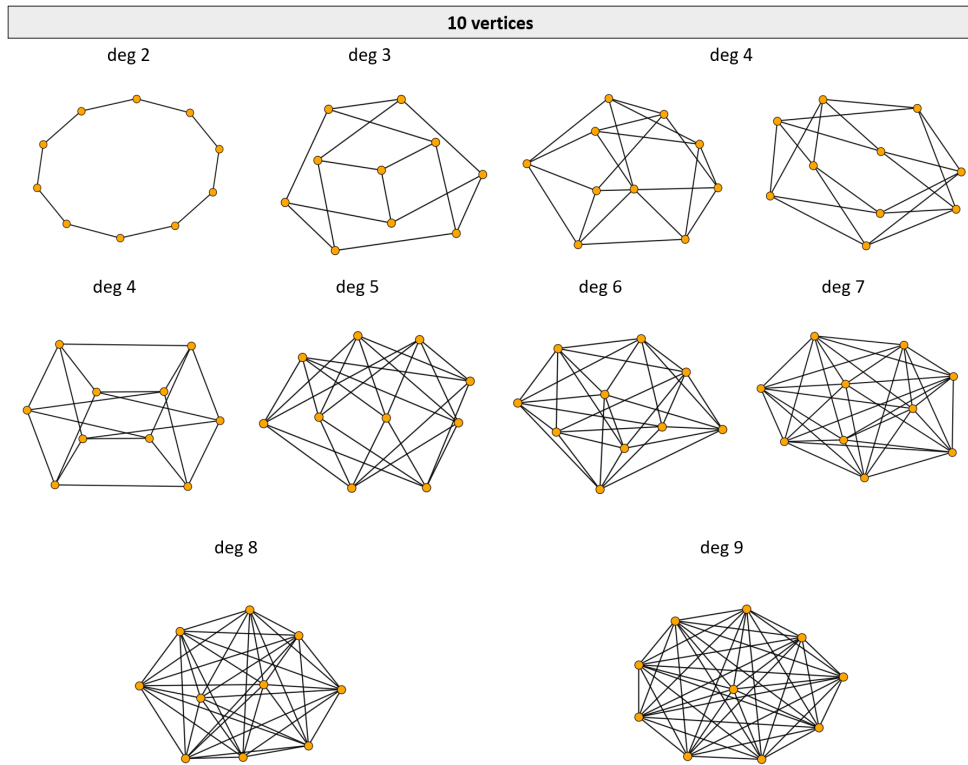
References

- [1] Shuliang Bai, Yong Lin, Linyuan Lu, Zhiyu Wang, and Shing-Tung Yau. Ollivier Ricci-flow on weighted graphs. October 2020.
- [2] Yann Ollivier. A visual introduction to riemannian curvatures and some discrete generalizations. 2012.
- [3] Adam Gosztolai and Alexis Arnaudon. Unfolding the multiscale structure of networks with dynamical ollivier-ricci curvature. 2021.
- [4] M. Meringer. Fast generation of regular graphs and construction of cages. *Journal of Graph Theory* 30, pages 137–146, 1999.
- [5] A. Steger and N. Wormald. Generating random regular graphs quickly. *Probability and Computing*, pages 377–396, 1999.
- [6] Allen Tannenbaum Romeil Sandhu, Tryphon Georgiou. Market fragility, systemic risk, and ricci curvature, 2015.
- [7] Vinh Nguyen Nhat Ho Stanley Osher Tan Nguyen Khang Nguyen, Hieu Nong. Revisiting over-smoothing and over-squashing using ollivier-ricci curvature, 2023.
- [8] Jie Gao Xianfeng David Gu Chien-Chun Ni, Yu-Yao Lin and Emil Saucan. Ricci curvature of the internet topology, 2015.
- [9] W. W. Zachary. An information flow model for conflict and fission in small groups. *Journal of Anthropological Research* 33, pages 452–473, year = 1977,.
- [10] Chien-Chun Ni, Yu-Yao Lin, Feng Luo, and Jie Gao. Community detection on networks with ricci flow. *Scientific reports*, 9(1):1–12, 2019.
- [11] Rémi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aurélie Boisbunon, Stanislas Chambon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, Léo Gautheron, Nathalie T.H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. Pot: Python optimal transport. *Journal of Machine Learning Research*, 2021.
- [12] Duncan J. Watts and Steven H. Strogatz. Collective dynamics of small-world networks. *Nature*, page 440–442, 1998.

6 Appendix

6.1 Appendix I: Regular graphs of constant curvature up to 11 vertices





6.2 Appendix II: Random graphs with constant curvature

```

    "Detect regular graphs with constant curvature"

#Clean file
e=open("/content/drive/MyDrive/adj21.txt", "r+")
for line in e:
    if line[0].isdigit() == True:
        edge[cont].append(line)
    else:
        if line[0] == 'G':
            print(cont)
        else:
            cont=cont+1

#Compute curvature
for i in range(0,size,2):
    sum=0
    G = nx.parse_adjlist(edge[i], nodetype=int) #create graph with given edge list
    for n1,n2 in list(G.edges()):
        orc = OllivierRicci(G, alpha=0.2) #compute ricci curvature
        orc.compute_ricci_curvature()
        G_orc = orc.G.copy()
        sum=sum + (G_orc[n1][n2]["ricciCurvature"])

x=[e for e in G.edges]
dif1=abs(sum - G_orc[x[0][0]][x[0][1]]["ricciCurvature"]*len(x))
#if the sum of the curvature of the edges is the same as the curvature of an
#edge times the number of edges then
#the graph is a potential candidate
if ((dif1 < 0.01) == True and (dif2 < 0.01) == True ) :
    print("Found= ",i, len(x), sum, G_orc[x[0][0]][x[0][1]]["ricciCurvature"])

```

6.3 Appendix III: Experimental analysis of non-convergence

```

    "How the weight changes for every iteration of the Ricc Flow"

#Initialize Ricci Flow
G1=G #make a copy of the original graph
orf = OllivierRicci(G, weight="weight", alpha=0.1, proc=4, verbose="INFO")
number_of_lists = G.number_of_edges()
flow = [[] for i in range(number_of_lists)]
iter=20 #set number of iterations
num= [ ]
for i in range(iter):
    orf.set_verbose("ERROR")
    orf.compute_ricci_flow(iterations=i)
    G_rf = orf.G.copy()
    weights = nx.get_edge_attributes(G_rf, "weight").values()
    weight_list=list(weights)
    weights=np.array(weight_list)
    for j in range(number_of_lists):
        flow[j].append(weights[j])
    num.append(i)
    i=0
    G=G1 #reset G to the original graaph
#Plot results
for n1,n2 in list(G.edges()):
    plt.plot(num, flow[i], label=(n1, n2))
    i=i+1
plt.xlabel('Iteration')

```

```
plt.ylabel('Weight')
plt.legend(loc='lower right')
```

```
"3D Plot of the change in the weights throughout the Ricci Flow in function
of the idleness"

number_of_lists = G.number_of_edges()
flow = [ ]
iter=10 #Set number of iterations
p=0.1 #set alpha
num= [ ]
para=[ ]
G1=G #copy of the original graph
for j in range(14):
    orf = OllivierRicci(G, alpha=p, proc=4, verbose="INFO")
    for i in range(iter):
        orf.set_verbose("ERROR")
        orf.compute_ricci_flow(iterations=i)
        G_rf = orf.G.copy()
        weights = nx.get_edge_attributes(G_rf, "weight").values()
        weight_list=list(weights)
        weights=np.array(weight_list)
        flow.append(weights[3]) #choose to plot a specific edge (i.e edge 3)
        para.append(p)
        num.append(i)
    G=G1 #reset to original graph
    p=p+1/20

#plot results
plt.rcParams["figure.figsize"] = [7, 7]
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot_trisurf(para,num,flow, cmap='viridis', edgecolor='none');
ax.set_ylabel('Iteration', fontsize=10)
ax.set_xlabel('Idleness: p', fontsize=10)
ax.set_zlabel('Weight', fontsize=10)
ax.view_init(elev=35., azimuth=-45, roll=0)
plt.show()
```

6.4 Appendix IV: Robustness analysis

```
"Robustness analysis using normal distribution truncated"

#Initialize graph using Watts Strogatz model
G = nx.watts_strogatz_graph(200,10, 0.2, seed=None)
Gcc = sorted(nx.connected_components(G), key=len, reverse=True)
G = G.subgraph(Gcc[0])
pos = nx.spring_layout(G)
nx.draw_networkx(G, pos, node_color='green', alpha=0.5, font_size =0, node_size=
20)

G0=G
iter=10
desv=0
value=[ ] #value of standard derviation
acuracy1=[ ]
for k in range(0,1):
    G1=G0
    i=0
    x=[ ]
```

```

desv=desv+0.1
value.append(desv)
X = get_truncated_normal(mean=0, sd=desv, low=-0.9, upp=number_of_lists ) #get
truncated normal

number_of_lists = G1.number_of_edges()
x.append(X.rvs(number_of_lists))
weight=np.full((1,number_of_lists), 1)
weight=np.ones(number_of_lists) #all the weights are set up to 1 as unweighted
graph

weight= x[0]+weight #weights with noise
flow = [[] for i in range(number_of_lists)]
num= [ ]
for n1,n2 in list(G1.edges()): #update weights with the noise on the grapg
    G1.edges[(n1,n2)]['weight'] = weight[0][i]
    i=i+1
orf = OllivierRicci(G1, weight="weight", alpha=0.2, base=math.e, exp_power=1,
proc=4, verbose="INFO") #initialize
Ricci Flow

for i in range(iter):
    G1=G0
    orf.set_verbose("ERROR")
    orf.compute_ricci_flow(iterations=i)
    G_rf = orf.G.copy()
    weights = nx.get_edge_attributes(G_rf, "weight").values()
    weight_list=list(weights)
    weights=np.array(weight_list)
    for j in range(number_of_lists):
        flow[j].append(weights[j])
    num.append(i)
predicted=[ ]
for i in range(0,number_of_lists): #classify in groups 0 or 1
    if flow[i][iter-1] <= 1:
        predicted.append(0)
    else:
        predicted.append(1)
accuracy1.append(metrics.accuracy_score(actuals, predicted))

#Plot Results
plt.scatter(value,accuracy1, label="Step Function", c='purple')
plt.title("Acuracy Score of the weight calssification after Ricci Flow")
plt.xlabel('Standard Deviation')
plt.ylabel('Acuracy')
plt.ylim(0.95,1.05)
plt.legend(loc='lower right')

```